Now we C++

- By Aditya Varma

Preface

Welcome to Let's Learn C++

This book is written to help you learn C++ as a skill, not as a subject.

C++ is a language that gives you both control and structure. It allows you to work close to how programs actually run in memory, while also providing powerful tools to design large, organized systems. Because of this balance, learning C++ builds a deep understanding of programming that makes other languages easier to grasp.

The chapters in this book are arranged carefully. Each concept is introduced only when it is needed, and every idea is supported by simple programs that explain why something works, not just how to write it.

C++ can feel vast at first. It has many features, and it does not force you into one way of thinking. This book simplifies that journey by guiding you step by step, starting from the basics and gradually moving toward structured and object-oriented programming.

No prior programming knowledge is assumed. You are expected only to read, practice, and stay curious. This book is meant to be read slowly, like a guide. Type the programs. Modify them. Break them. Observe what changes.

Take your time. Ask questions. Make mistakes. That is how real understanding is built.

-Aditya Varma

License & Credits

Credits:

Certain icons, illustrations, and portions of reference content are adapted from publicly available educational platforms, including the Coding X mobile application. Proper credit is given where applicable. All original rights remain with their respective creators, and this material is used solely for educational purposes.

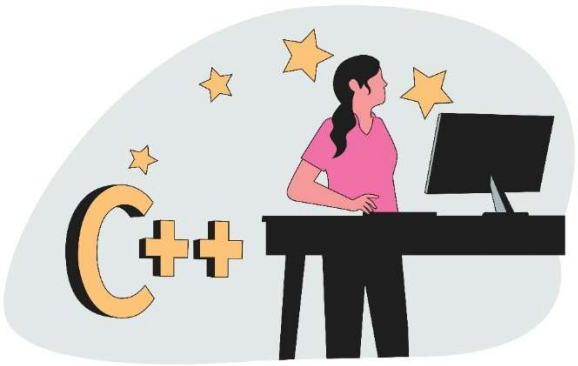
Index

Getting Started with C++

C++ is a general-purpose programming language that sits at an interesting intersection: it gives you the power to work close to the machine while also letting you build large, structured, and maintainable software. It is often described as an extension of C, but over time it has grown into a language of its own, supporting multiple programming styles including procedural, object-oriented, and even generic programming.

At its core, C++ is about **control and efficiency**. When you write a C++ program, you are not just telling the computer *what* to do, but also influencing *how* it does it. You can manage memory directly, control performance, and decide how data is stored and accessed. This makes C++ both powerful and demanding. It does not hide complexity from you, but instead gives you the tools to understand and handle it.

Because of this, learning C++ is like learning how the engine of a machine works rather than just driving it. Once you understand it, other languages start to feel more intuitive.



History Of C++:

Like other programming languages popular today, C++ has quite a story behind it, stretching back to 1979. **Bjarne Stroustrup**, a Danish computer scientist working at Bell Labs, started work on a new programming language called **'C with Classes'** deriving from the original C language created by Dennis Ritchie

A large inspiration for the language was Simula, which is considered to be the first object-oriented programming language ever. While Stroustrup was programming for his Ph.D. thesis, he found Simula pleasant to program in, but the language was too slow for practical use. Therefore, he set out to add Simula-like features to C, thus making a language that was both performant and relatively high-level for the time. In this way, the 'C with Classes' programming language was born. Initially, C with Classes just added features to the C compiler.

In 1982, Stroustrup started to build the next version of his language. He built a new compiler based on the existing C compiler and added a ton of new features. After cycling through a couple of names, he ended up with 'C++'. The name comes from the C's incrementation operator, it means C plus one. In 1985, the first commercial implementation of C++ was released. The same year also saw the release of the first reference book of the language - The C++ Programming Language written by the creator himself

Stroustrup wanted the addition of object-oriented programming (OOP), which drove him to create C++. Therefore, the inclusion of OOP is the significant difference between C and C++. To summarize his interview with Big Think, the motivation for making C++ was increasing a program's maintainability.

To Conclude:

Although C++ is used in many industries and can be used to write almost anything, it particularly excels in delivering performance and using system resources efficiently. As we have said, it's an extension of the C programming language, therefore it retains a strong link to C, and will compile most C programs. Isn't that WONDERFUL?

Oh! That was quite a theory and a flashback into C++'s history. But, before you learn anything, you should know how it came into existence. From here on, we will move ahead to the more technical side and slowly start writing some code.

Difference Between C and C++:

To understand C++, it helps to see where it comes from. C is a procedural programming language, meaning it focuses on functions and step-by-step instructions. Programs in C are typically organized around functions that operate on data.

C++, on the other hand, expands this idea by introducing **object-oriented programming (OOP)**. Instead of only thinking in terms of functions, you can organize programs around **objects**, which combine data and behaviour together. This makes it easier to model real-world systems and manage large codebases.

Another key difference is that C++ provides additional features such as classes, inheritance, polymorphism, and function overloading. These features allow developers to write more modular and reusable code. While C gives you the raw tools, C++ adds layers that help you structure complex programs more effectively.

Despite these differences, C++ still retains most of C's capabilities. This means you can write low-level, efficient code just like in C, while also taking advantage of higher-level abstractions when needed. In a way, C++ is like a toolkit that includes both basic tools and advanced machinery, and it is up to the programmer to decide what to use.

Features of C++:

A lot of developers in different communities argue about the best programming language. Actually, it is impossible to identify which tool or technology is indeed the optimum variant for ALL problems since different languages are designed for different kinds of problems.

Nevertheless, the C++ language has some powerful arguments up its sleeve. In this section, we will look at some of the features of C++ and try to understand what makes it so popular and a top choice for various projects.

- **Performance**: Performance is directly related to speed. When it comes to speed, C++ has no match in today's list of popular languages. C++ is the Flash of the world of programming!

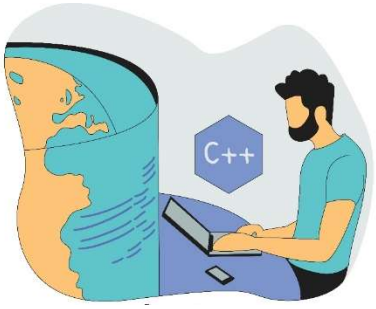
  It is one of the fastest and most predictable languages out there, only contested by other low-level programming languages like Rust. The control provided by C++ over the system resources enables a skilled coder to write a program that is quicker and more powerful than a similar program written in another programming language.

  Interesting Fact: Most AAA game titles are written using C++ because these titles push the limits of existing hardware, so resource efficiency is very important.

- **Flexibility**: C++ is a multi-paradigm programming language. This means that it supports other styles such as procedural programming, in addition to object-oriented programming.

  These paradigms are essentially different ways of looking at and solving a problem. Two different C++ programmers can solve the same problem in different ways. Paradigms can also be combined to get the most efficient result.

  On the other side, having different ways of solving problems makes C++ more complex, but at the same time, more powerful as well.

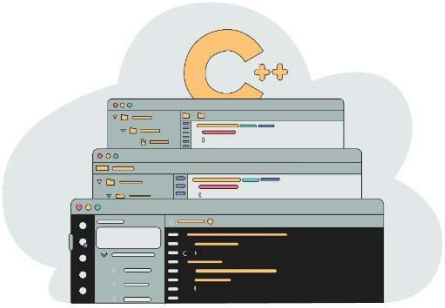


- **Syntax**: When it comes to syntax, it is a bit harder to choose the best option. Today, the majority of popular programming languages have pretty nice syntax. Unlike some other languages, C++ has quite an intuitive syntax, which is actually not so hard to comprehend, by looking at the code a bit more creatively.

  Hence, the prize named "Easiest Syntax" doesn't go to C++, since it has more elements to write in contrast to some other languages. Fortunately, even with more code and intuitive syntax, C++ code is more structured and follows the same set of rules. Therefore, despite its length, C++ has a pretty straightforward syntax.

- **Large Ecosystem**: The language has been around for almost 40 years, which means that most of the software problems have already been solved by open-source libraries and frameworks. C++ has a ton of developers that use it, upgrade it, and write open-source libraries.

  Therefore, in simple words, while learning or using the language, you can draw on the work done by these people already.



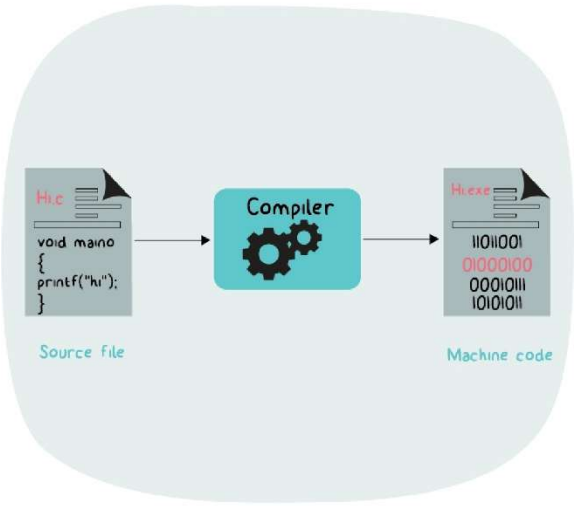
C++ is an exceptional language when it comes to its abilities. It lets us do virtually anything possible and impossible. It is a technology that can be used to accomplish a tremendous variety of tasks.

A large number of jobs available is a reflection of the long-standing popularity of C++. Not only do many new applications get coded in C++, but the huge range of programs already built during the language necessitate hiring coders to keep them current and updated. Hence, C++ developers are welcome to nearly every software development company.

How C++ Works?

When you write a C++ program, you do not directly communicate with the computer's hardware. Instead, you write code in a human-readable form called **source code**, usually saved in files with a .cpp extension. This code must be translated into a form that the computer can understand and execute.

This translation happens through a process called **compilation**.



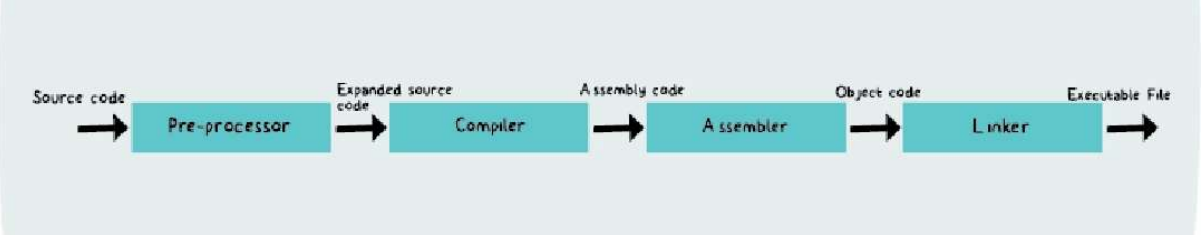
Compiled languages are converted directly into machine code that the processor can execute. As a result, they tend to be faster and more efficient to execute than interpreted languages.

Steps in Compilation:

The first step involves the compiler taking your source code and checking it for errors. If the code is valid, it is converted into an intermediate form called **object code**. This object code is not yet a complete program, but it contains machine-level instructions.

In the next step, a **linker** combines this object code with other necessary pieces, such as libraries, to produce the final **executable file**. This executable file is what you actually run on your system.

So, the journey of a program can be thought of as a transformation:



Where is C++ used?

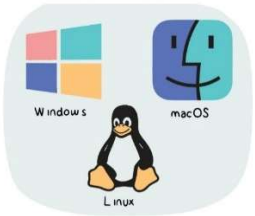
Remember, C++ is a general-purpose programming language? This means it is used in most places! And that's the reason why C++ has held its grip even after 35+ years of its release. C++ can be used for; Operating Systems Development, Game Development, Computer Graphics, Cloud/Distributed Systems, Compiler Development, And more... The list goes on! Let's dive deep.

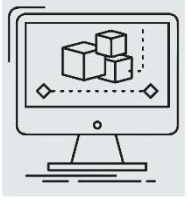
- **Operating Systems:** Be it Microsoft Windows or Mac OSX or Linux - all of them are programmed in C++. No doubt, right?

  Speed and security are the main factors for an OS, and C++ is the master of those features, C/C++ is the backbone of all well-known operating systems owing to the fact that it is a strongly typed and fast programming language which makes it an ideal choice for developing an operating system.
- **Game Development**: Game Development is one of the fields that is said to have one of the brightest futures in the upcoming decade. Well, believe it or not, C++ is a direct ticket to a billion-dollar game development industry.

  To tackle big games in the larger gaming companies, knowing C++ is critical. It's fast, the compilers and optimizers are solid, and you get a lot of control over memory management. It has extensive libraries, which come in handy for designing and powering complex graphics.
- **Computer Graphics**: All graphics applications require fast rendering and just like the case of web browsers, C++ helps in reducing the latency. Software that employs computer vision, digital image processing, high-end graphical processing - they all use C++ as the backend programming language.

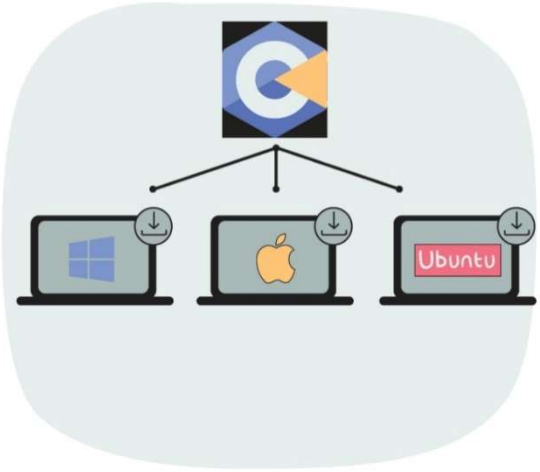
  Even the popular games that are heavy on graphics use C++ as the primary programming language. The speed that C++ offers in such situations helps the developers in expanding the target audience because an optimized application can run even on low-end devices that do not have high computation power available.
- **Cloud/Distributed Systems**: Large organizations that develop cloud storage systems and other distributed systems also use C++ because it connects very well with the hardware and is compatible with a lot of machines. Cloud storage systems use scalable file systems that work close to the hardware.
- **Compiler Development**: A compiler is responsible for not only converting the source code to machine code but also performs various checks to ensure the code is correct and error-free. The compilers of various programming languages use C and C++ as the main programming language.




Installing C++ and its IDE:

To work with C++ on your machine, first, you will need to install the C++ Compiler in your system. It is unlikely that your system will be shipped with C++ Compiler.



Almost every C++ programmer uses the simplest way for getting this by using the GCC compiler. But first, would you like to know what GCC is? GCC stands for GNU Compiler Collection, it is an open-source compiler that supports the execution of multiple programming languages like C, C++, Fortran, Golang, and more.

GCC has been ported to multiple platforms and hence it helps in building C++ programs on a wide variety of systems. Currently, GCC's appropriate version for multiple platforms is available widely and for free.

But we are going to use something more efficient i.e., “Codeblocks”.

Installing Code::Blocks with MinGW:

Before we start writing C++ programs, we need a place to write and run them. For this, we will use Code::Blocks, which comes with a built-in C/C++ compiler called MinGW.
Follow the steps carefully.

Step 1: Download Code::Blocks

- Open your browser
- Search for: Code::Blocks official website
- Open the website:
  👉 Code::Blocks
- Go to the Downloads section
- Click on “Download the binary release”

Step 2: Choose the Correct Version

You will see multiple options. Choose the one that includes MinGW.

- Look for a file named similar to:

  codeblocks-20.03mingw-setup.exe

⚠️ Important:

- Always choose the version with "mingw" in the name
- This version already contains the C compiler
- No need to install anything separately

Step 3: Install Code::Blocks

- Double-click the downloaded .exe file
- Click Next → I Agree → Next
- Keep default settings
- Click Install

  Once installation completes:
- Click Finish

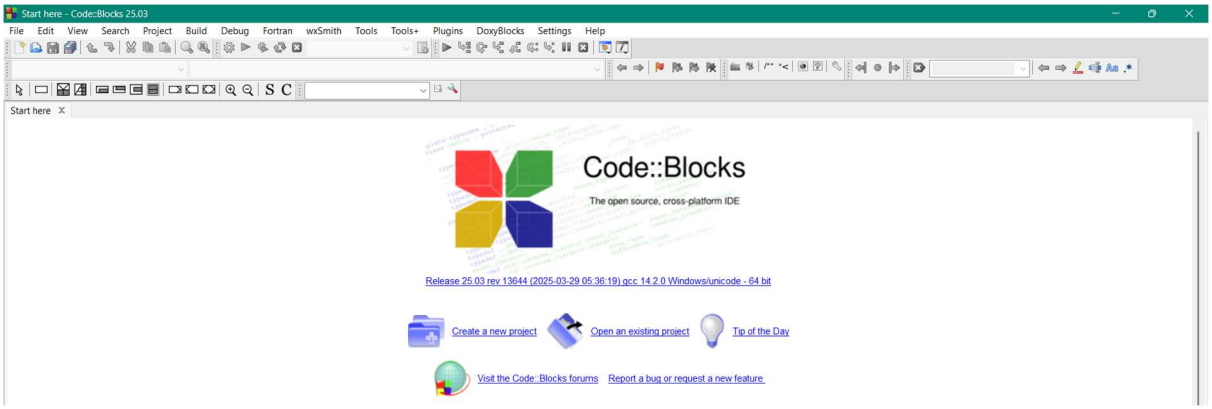
Step 4: First Launch Setup

- Open Code::Blocks
- It may ask to select a compiler

If prompted:

- Choose: GNU GCC Compiler
- Click Set as default
- Click OK

The setup is done and screen will look like this:



Creating files in Code::Blocks:

Step 1: Create Your First Project

- Click on Create a new project
- Select Console Application
- Click Go → Next
- Choose language: C++
- Click Next

Step 2: Set Project Details

- Enter a Project Title (e.g., MyFirstProgram)
- Choose a folder location
- Click Next → Finish

Step 3: Write Your First Program
You will see a default file (main.cpp).

- Inside it, you can write your first program
- Or modify the existing code

Step 4: Build and Run the Program

- Click on Build and Run (green play button)
  OR press F9
- Output will appear in a console window

At this stage, do not worry about:

- How the compiler works internally
- Complex settings
- Advanced configurations

Right now, your goal is simple:
Write → Run → Observe

First C++ Program & Boilerplate

Let's begin with the classic first step into any programming language. It may look small, but it opens the door to everything that follows.

```cpp
#include <iostream>

int main() {
    std::cout << "Hello, World!";
    return 0;
}
```

When this program runs, it simply prints:

Hello, World!

At first glance, this might feel like a few strange symbols stitched together. But each line has a purpose, and together they form the basic skeleton of almost every C++ program you will write.

Understanding the Structure of a C++ Program:

A C++ program is not just random lines of code. It follows a structure, almost like a recipe. Each part has a role, and removing or misplacing one piece can break the whole program.

Let's gently unwrap each component.

1. #include <iostream>: Bringing in Tools

This line is a **preprocessor directive**. It tells the compiler to include a file before the program is compiled. The file iostream stands for **input-output stream**. It contains definitions for objects that allow your program to take input and display output.

Think of it like opening a toolbox before starting work. Without it, your program would not know how to print anything on the screen.

2. What is iostream?

iostream is a standard library file in C++ that provides functionality for input and output operations.

It gives you access to:

- cout → for output (printing to screen)
- cin → for input (taking data from user)

Without including iostream, using cout or cin would result in an error because the compiler would not recognize them.

3. What is std?

In C++, many standard features are grouped under something called a **namespace**. The standard library belongs to the namespace called std (short for *standard*).

So, when you write: std::cout you are telling the compiler: "Use cout from the standard namespace."

This avoids confusion in large programs where different libraries might have functions or objects with the same name.

4. int main(): The Starting Point

Every C++ program begins execution from the main() function.

```
int main() {
   // code
}
```

- main is the entry point of the program
- int means the function returns an integer value
- The {} braces contain the body of the program

No matter how big your program becomes, execution always starts from here.

5. cout: Displaying Output

Inside the main() function, we wrote: std::cout << "Hello, World!";

Here:

- cout stands for **console output**
- << is the insertion operator (it sends data to output)
- "Hello, World!" is the text to display

So this line simply tells the computer: "Print this message on the screen."

6. return 0; Ending the Program

At the end of main(), we often write: return 0;

This indicates that the program has executed successfully.

- 0 typically means success
- Any non-zero value can indicate an error

While modern compilers may allow you to omit it, it is good practice to include it for clarity.

Making It Simpler: Using namespace std

Writing std::cout every time can feel repetitive, especially for beginners. C++ allows you to simplify this using: using namespace std;

This tells the compiler: "Use the standard namespace by default."
Now you can write a cleaner version of the same program.

C++ Boilerplate (Basic Template):

```
1  #include <iostream>
2
3  int main() {
4      // Program codes goes here
5      return 0;
6  }
```

This is the **basic boilerplate** of a C++ program.

Final Thought:

If a C++ program were a stage play:

- #include opens the backstage doors
- iostream brings in the actors for input/output
- std tells you which troupe they belong to
- main() is where the play begins
- cout is the actor speaking to the audience

Right now, it's just one line of dialogue. Soon, you'll be directing entire performances.

Escape Sequences & Comments

// Comments in C++:

When you write programs, you are not just communicating with the computer, but also with yourself and others who may read your code later. This is where **comments** come in. Comments are lines in a program that are ignored by the compiler. They do not affect how the program runs, but they make the code easier to understand.

In real-world programming, code is rarely written once and forgotten. It is read, modified, debugged, and improved over time. Comments act like small notes left in the code, explaining *why something is done*, not just *what is written*. A well-commented program feels guided, while a program without comments can feel like walking through a maze without signs.

C++ provides two types of comments:

- **Single-line comments**, which start with // and continue till the end of the line
- **Multi-line comments**, which start with /* and end with */, allowing you to write comments across multiple lines

Comments are especially useful for:

- Explaining logic
- Temporarily disabling code during testing
- Making programs readable for others

Lesson Program: Comments in C++

Source Code: Comments.cpp

```cpp
#include <iostream>
using namespace std;

int main() {

    // Printing a welcome message
    cout << "Welcome to C++ Programming!\n";

    // Printing another line
    cout << "This program demonstrates comments usage.\n";

    /*
        Multi-line comment:
        The following lines perform simple addition
        and display the result
    */

    int a = 10;   // First number
    int b = 20;   // Second number
    int sum = a + b;   // Adding numbers

    // Displaying the result
    cout << "Sum = " << sum << endl;

    return 0;   // Program ends successfully
}
```

// Escape Sequence:

When printing text, sometimes we need more control than just plain words. We may want to move to a new line, insert a tab space, print quotation marks, or format the output neatly. This is where **escape sequences** come into play.

An escape sequence is a combination of characters that begins with a backslash \. It tells the compiler to perform a special action instead of printing the characters as they are. Think of escape sequences as hidden instructions embedded inside strings. They allow you to control how the output appears on the screen.

Why Do We Need Escape Sequences?

If you write: `cout << "Hello World";` everything appears on one line.
But what if you want:

```
Hello
World
```

You cannot just press Enter inside the string. Instead, you use:

```
cout << "Hello\nWorld";
```

Here, `\n` tells the program to move to a new line.

Where and How Are They Used?

Escape sequences are mainly used inside **string literals** (text inside quotes " "). They modify how the output is displayed.

For example:

- Formatting output
- Printing special characters
- Structuring readable console output

List of Escape Sequences in C

Here are the most useful escape sequences:

- `\n` → New line
- `\t` → Horizontal tab (space gap)
- `\b` → Backspace (removes previous character)
- `\r` → Carriage return (moves cursor to beginning of line)
- `\\` → Prints a backslash \
- `\"` → Prints double quotes "
- `\'` → Prints single quote '
- `\0` → Null character (end of string)

Lesson Program: Escape Sequence in C++
Source Code: EscapeSequence.cpp

```cpp
#include <iostream>
using namespace std;

int main() {

    // Demonstrating Escape Sequences in C++

    cout << "Demonstrating Escape Sequences in C++\n\n";

    // 1. New Line
    cout << "1. New Line (\\n):\n";
    cout << "Hello\nWorld\n\n";

    // 2. Tab Space
    cout << "2. Tab Space (\\t):\n";
    cout << "Hello\tWorld\n\n";

    // 3. Backspace
    cout << "3. Backspace (\\b):\n";
    cout << "Helloo\b World\n\n";

    // 4. Carriage Return
    cout << "4. Carriage Return (\\r):\n";
    cout << "Hello World\rHi\n\n";

    // 5. Backslash
    cout << "5. Backslash (\\\\):\n";
    cout << "\\\n\n";

    // 6. Single Quote
    cout << "6. Single Quote (\\'):\n";
    cout << "\'\n\n";

    // 7. Double Quote
    cout << "7. Double Quote (\\\"):\n";
    cout << "\"C++\"\n\n";

    // 8. Alert (may produce sound)
    cout << "8. Alert (\\a):\n";
    cout << "\a\n";

    return 0;    // Program ends
}
```

The Language Building Blocks: Tokens

Before a C++ program can be understood by the compiler, it must be broken down into smaller pieces. These smallest individual units of a program are called **tokens**. Just like a sentence is made up of words and punctuation, a C++ program is made up of tokens. The compiler reads your code not as full lines or paragraphs, but as a sequence of these meaningful building blocks.

Whenever you write a program, whether it is a simple cout statement or a complex function, everything you type is ultimately divided into tokens. This is why understanding tokens helps you understand how the compiler "sees" your code.

In simple terms, **a token is the smallest unit in a C++ program that has meaning to the compiler**.

Why Are Tokens Important?

Tokens are important because they form the basic structure of any program. If tokens are used incorrectly, the compiler will not understand the code and will generate errors. For example, writing a variable name incorrectly or misusing an operator can break the entire program.

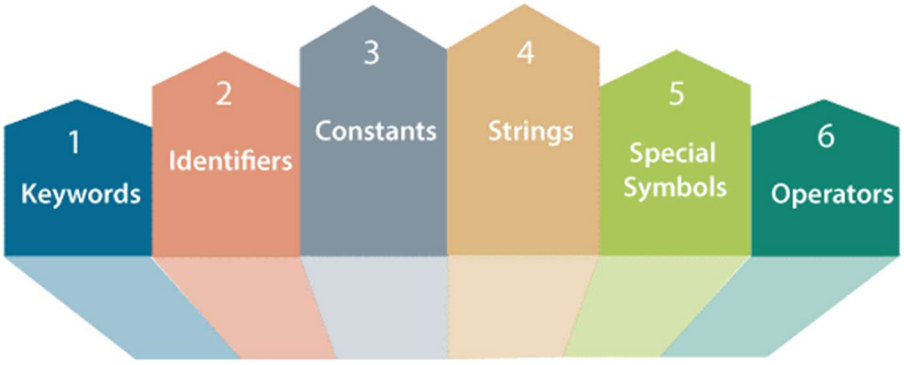
By learning tokens, you begin to see programming not as random syntax, but as a well-organized system where every symbol has a purpose. It also helps in debugging, because you can identify exactly which part of the code is causing an issue.

Types of Tokens in C++:

C++ mainly has **six types of tokens**, each playing a specific role:

- **Keywords**: Reserved words with special meaning (e.g., int, return, if)
- **Identifiers:** Names given to variables, functions, etc. (e.g., sum, main)
- **Constants (Literals):** Fixed values (e.g., 10, 3.14, 'A')
- **Strings**: Sequence of characters inside quotes (e.g., "Hello")
- **Operators**: Symbols that perform operations (e.g., +, -, *, =)
- **Punctuators (Special Symbols)**: Symbols used for structure (e.g., ;, {}, ())

Together, these tokens form every valid C++ program.



Character Set of C++

To form tokens, C++ uses a defined set of characters known as the **character set**. These are the basic symbols that you can use to write any program.

The C++ character set includes:

- Uppercase: A to Z
- Lowercase: a to z
- Digits: 0 to 9
- Special characters: ! " # $ % & ' ( ) * + - . : , ; ` ~ = < > { } [ ] ^ _ \ /
- Other special characters: Blank space, tab, etc. (Compiler ignores white spaces unless they are part of string)

These characters combine to form tokens, and tokens combine to form complete programs.

// Keywords & Identifiers:

In every C++ program, the words you write are not all treated the same. Some words are already defined by the language itself, while others are created by the programmer. This brings us to two important types of tokens: **keywords** and **identifiers**.

C++ Keywords:

Keywords are **reserved words** that have a fixed meaning in C++. They are predefined by the language and cannot be used for any other purpose, such as naming variables or functions.

For example, words like int, float, return, and if are keywords. Each of these has a specific role. When you write int, the compiler immediately understands that you are declaring an integer variable. These meanings are built into the language and cannot be changed.

C++ has a larger set of keywords compared to C because it supports advanced features like object-oriented programming. All keywords are written in lowercase and must be used exactly as defined.

Keywords act like the grammar of the language. They give structure to your program and tell the compiler how to interpret different parts of the code.

List of Keywords:

asm	auto	bool	break	case	catch
char	class	const	const_cast	condition	default
delete	do	double	dynamic_cast	else	enum
explicit	export	extern	false	float	for
friend	goto	if	inline	int	long
mutable	namespace	new	operator	private	protected
public	register	reinterpret_cast	return	short	signed
sizeof	static	static_cast	struct	switch	template
this	throw	true	try	typedef	typeid
typename	union	unsigned	using	virtual	void
volatile	wchar_t	while			

C++ Identifiers:

Identifiers are the names given to elements created by the programmer. These include variables, functions, arrays, classes, and more.

For example: int age; float height;

Here, age and height are identifiers. They are chosen by the programmer to represent data in a meaningful way. Unlike keywords, identifiers do not have predefined meanings. Their purpose depends entirely on how they are used in the program. Good identifiers make code more readable and easier to understand.

Rules for Defining Identifiers in C++:
While identifiers give you flexibility, there are certain rules that must be followed:

1. An identifier must begin with a letter (A–Z or a–z) or an underscore (_)
2. It can contain letters, digits (0–9), and underscores
3. It cannot start with a digit
4. It cannot be a keyword
5. Identifiers are case-sensitive (age and Age are different)
6. No special symbols like @, #, %, or spaces are allowed

✔ Example Valid Identifiers:

- sum, totalMarks, _count, Age1

❌ Example Invalid Identifiers:

- int (keyword), 2ndValue (starts with digit), my-name (special character)

Lesson Program: Keywords and Identifiers in C++
Source Code: KeywordsAndIdentifiers.cpp

```cpp
#include <iostream>
using namespace std;

int main() {

    // int, float, char are keywords
    // age, height, grade are identifiers

    int age = 23;          // 'int' is a keyword, 'age' is an identifier
    float height = 5.9;    // 'float' is a keyword, 'height' is an identifier
    char grade = 'A';      // 'char' is a keyword, 'grade' is an identifier

    // cout and endl are part of standard library (used for output)

    cout << "Age: " << age << endl;
    cout << "Height: " << height << endl;
    cout << "Grade: " << grade << endl;

    // return is a keyword
    return 0;
}
```

// Constants and Variables in C++

Every C++ program works with **data**. Data is the information that a program stores, processes, and displays. In C++, this data is mainly classified into two types: **constants** and **variables**.

Constants in C++:

A constant is a **fixed value** that does not change during the execution of a program. Once defined, its value remains the same throughout. Constants are used when certain values are meant to stay unchanged, such as mathematical values or fixed quantities.

Constants can be of different types:

- **Integer constants** → 10, -25, 0
- **Floating-point constants** → 3.14, -0.001, 2.5e3
- **Character constants** → 'A', 'z', '1'
- **String constants** → "Hello", "C++ Programming"

In C++, constants can also be defined using the const keyword.
Example:

```
const int days = 7;
```

Here, days is a constant whose value cannot be changed.

Variables in C++:

A variable is a **named location in memory** used to store data that can change during program execution. Unlike constants, variables can be modified as the program runs. Before using a variable, it must be declared with a data type, which tells the compiler what kind of data it will store.

Example:

```
int age = 20;
float height = 5.9;
char grade = 'A';
```

Here:

- age, height, and grade are variables
- Their values can change during execution

Variables are essential because they allow programs to store and manipulate dynamic data.

Rules for Variables

The rules for naming variables in C++ are the same as identifiers:

- Must begin with a letter or underscore
- Can contain letters, digits, and underscores
- Cannot be a keyword
- Case-sensitive
- No special symbols or spaces allowed

Lesson Program: Constants and Variables in C++
Source Code: ConstantsAndVariables.cpp

```cpp
#include <iostream>
using namespace std;

int main() {

    // Constant (value cannot be changed)
    const float PI = 3.14;

    // Variables (values can change)
    int days = 7;
    int age = 23;
    float height = 5.9;
    char grade = 'A';

    // Displaying constant value
    cout << "PI = " << PI << endl;

    // Displaying variable values
    cout << "Days in a week = " << days << endl;
    cout << "Age = " << age << endl;
    cout << "Height = " << height << endl;
    cout << "Grade = " << grade << endl;

    return 0;
}
```

// Constants, Strings, Operators, and Punctuators in C++:

In addition to keywords and identifiers, C++ programs are built using other important tokens such as **constants, strings, and punctuators**. These tokens help represent values, text, and the structure of a program. Each of them plays a simple but essential role in writing meaningful code.

Constants in C++:

Constants are **fixed values** that do not change during the execution of a program. Once a constant is defined, its value remains the same throughout.

For example: 10, 3.14, 'A'

These values are directly used in programs without needing a variable. Constants can be of different types:

- **Integer constants** → 10, 100
- **Floating-point constants** → 3.14, 2.5
- **Character constants** → 'A', 'b'

Constants make programs simple when values do not need to change. They are used to represent fixed data such as marks, grades, or predefined numbers.

Strings in C++:

Strings are a sequence of characters enclosed in **double quotes**.

For example: "Hello", "C++ Programming"

Strings are mainly used to display messages or text in a program. They are commonly used with output statements like cout. Unlike character constants (which use single quotes), strings always use double quotes and can contain multiple characters.

Operators in C++:

Operators are **symbols used to perform operations** on data. They tell the compiler to carry out specific tasks such as addition, comparison, or assignment.

For example: +, -, *, /, =

Operators are used to:

- Perform calculations
- Compare values
- Assign values

They are essential for writing logic in programs.

Punctuators in C++:

Punctuators are **special symbols** used to organize and structure a C++ program. They help the compiler understand how different parts of the code are separated and grouped.

Some common punctuators include: ; ( ) { } , [ ] .

For example:

- ; marks the end of a statement
- { } define a block of code
- ( ) are used in functions

Without punctuators, the structure of a program would be unclear, and the compiler would not be able to interpret it correctly.

Lesson Program: Other tokens in C++
Source Code: OtherTokens.cpp

```cpp
#include <iostream>
using namespace std;

int main() {

    // Constants
    int a = 10;          // 10 is an integer constant
    int b = 5;           // 5 is an integer constant
    char ch = 'A';       // 'A' is a character constant

    // String
    cout << "Welcome to C++ Programming\n";

    // Operators
    int sum = a + b;     // '+' is an operator
    int diff = a - b;    // '-' is an operator

    // Displaying values
    cout << "Sum = " << sum << endl;
    cout << "Difference = " << diff << endl;
    cout << "Character = " << ch << endl;

    // Punctuators used:
    // ;  -> end of statement
    // {} -> block of code
    // () -> function

    return 0;
}
```

Variables and Data Types in C++

Until now, programs were mainly printing messages. But real programs do much more than that. They **store values, use them, and sometimes change them over time**.

Think about simple real-life situations:

- A student has marks
- A shop has prices
- A program calculates a result

In all these cases, some values need to be stored and used later. A computer does not "remember" things like we do. Instead, it stores everything in its memory. This is where **variables** come into the picture.

What is a Variable?

A variable is a **named location in memory** where a value is stored. When a program runs, the computer uses its memory to store data. Instead of remembering memory locations (which are complex), we give them simple names. These names are called **variables**.

In simple terms: A variable is a **container that stores data**

Understanding Variables with Memory:

You can imagine memory like a collection of boxes 📦 :

- Each box has a **name** → variable
- Each box stores a **value**
- Each box has a **type** → defines what kind of data it can hold

For example: `int age = 21;`

Here:

- int → type (integer)
- age → variable name
- 21 → value stored

Declaring a Variable:

Declaration means telling the compiler: "I want to create a variable of this type with this name"

```
int age;
float salary;
char grade;
```

At this stage:

- Memory is reserved
- No value is stored yet

Assigning a Value:

Assignment means giving a value to a variable.

```
age = 21;
salary = 55000.0;
grade = 'A';
```

This stores the values in memory.

Initializing a Variable:

Initialization means declaring and assigning a value at the same time.

```
int year = 2025;
float pi = 3.14;
char section = 'B';
```

This is the most common and recommended way.

Where Are Variables Used?

Variables are used:

- To store input values
- To perform calculations
- To hold results
- To control program flow

Almost every program depends on variables.

Rules for Naming Variables:

Variable names follow the same rules as identifiers:

- Must start with a letter (a–z, A–Z) or underscore (_)
- Can contain letters, digits, and underscores
- Cannot start with a digit
- Cannot be a keyword
- No spaces or special symbols
- Case-sensitive (age and Age are different)

Making a Variable Constant

Sometimes, we do not want a value to change. In such cases, we use **constants**. C++ provides **two ways** to create constants:

1. Using const Keyword:

```
const float PI = 3.14;
```

- Value cannot be changed later
- Type-safe (follows data type rules)

2. Using #define (Preprocessor Constant):

```
#define DAYS 7
```

- Defined before compilation
- No data type is specified
- Replaces value everywhere in code

Difference (Basic Idea):

- const → behaves like a variable (recommended in C++)
- #define → simple text replacement

Lesson Program: Variables in C++

Source Code: Variables.cpp

```cpp
#include <iostream>
using namespace std;

// Preprocessor constant
#define DAYS 7

int main() {

    // Declaration
    int age;
    float salary;
    char grade;

    // Assignment
    age = 21;
    salary = 55000.0;
    grade = 'A';

    // Initialization
    int year = 2025;
    float pi = 3.14;
    char section = 'B';

    // Constant using const
    const float PI = 3.14159;

    // Using variables
    cout << "Age: " << age << endl;
    cout << "Salary: " << salary << endl;
    cout << "Grade: " << grade << endl;

    cout << "Year: " << year << endl;
    cout << "Pi: " << pi << endl;
    cout << "Section: " << section << endl;

    // Using constants
    cout << "Days in a week (#define): " << DAYS << endl;
    cout << "Value of PI (const): " << PI << endl;

    return 0;
}
```

// Data Types in C++

In C++, every variable stores some kind of data. But not all data is the same. Some values are whole numbers, some are decimal numbers, and some are characters or text. To handle this properly, C++ uses **data types**.

A **data type** defines:

- What kind of data a variable can store
- How much memory it will use
- What range of values it can hold

In simple terms: A data type tells the computer **what type of value is being stored**

Why Do We Need Data Types?

Computers manage memory very carefully. If you store a small number, it should not take too much space. If you store a large number or decimal value, more memory may be needed.

Data types help:

- Use memory efficiently
- Perform correct operations
- Avoid errors

For example, storing 10 is different from storing 10.5, and both are different from storing 'A'.

Classification of Data Types in C++

1. Primary (Fundamental) Data Types in C++:

Primary data types are the basic built-in types supported directly by the compiler.

They include:

- int → stores integers (e.g., 10, -5)
- float → stores decimal values (e.g., 3.14)
- double → stores decimal values with higher precision
- char → stores a single character (e.g., 'A')
- bool → stores logical values (true or false)

2. Derived Data Types:

Derived data types are formed using primary data types.

Examples:

- Arrays
- Pointers
- Functions

They help in handling collections of data and memory.

3. User-defined Data Types:

User-defined data types are created by the programmer.

Examples:

- struct
- union
- enum
- typedef

They help in organizing complex data.

Note on Size and Range

Each data type uses a specific amount of memory and has a defined range. This may vary depending on the system and compiler, but the purpose remains the same: Efficient storage and proper data handling

Lesson Program: Data Types in C++

Source Code: DataTypes.cpp

```cpp
#include <iostream>
using namespace std;

int main() {

    // Primary data types

    int age = 23;               // Integer type
    float pi = 3.14;            // Float type
    double big = 123456.789;    // Double type
    char grade = 'A';           // Character type
    bool isStudent = true;      // Boolean type

    // Displaying values
    cout << "Integer (age): " << age << endl;
    cout << "Float (pi): " << pi << endl;
    cout << "Double (big): " << big << endl;
    cout << "Character (grade): " << grade << endl;
    cout << "Boolean (isStudent): " << isStudent << endl;

    return 0;
}
```

// Type Modifiers and Type Casting

Once you understand data types, the next step is learning how to **adjust and control them**. Sometimes you need to convert one type into another, and sometimes you need to expand or restrict the range of a data type. That's where **type casting** and **type modifiers** come in.

Type Casting in C++

What is Type Casting?

Type casting means **converting one data type into another**. In real life, imagine you have a bill amount **₹99.75**, but you only want to store **₹99** as a whole number. You are converting a decimal into an integer. This is exactly what type casting does in C++. It changes the **type of a value** so it can be used differently.

Why Do We Need Type Casting?

Type casting is used:

- To perform accurate calculations
- To avoid unintended results
- To match data types in expressions

Example:

```
int a = 10, b = 3;
float result = a / b;
```

This gives 3 (wrong for decimal division).

Correct way:

```
float result = (float)a / b;
```

Now the result is 3.33.

Types of Type Casting:

1. Implicit Type Casting (Automatic)

Done automatically by the compiler.

```
int a = 10;
float b = a;   // int → float
```

2. Explicit Type Casting (Manual)

Done by the programmer.

```
float x = 5.7;
int y = (int)x;   // float → int
```

Type Modifiers in C++

What are Type Modifiers?

Type modifiers are used to **modify the size or range of basic data types**.

Think of them like choosing different container sizes:

- Small values → short
- Large values → long
- Only positive values → unsigned

Why Do We Need Type Modifiers?

Because different data needs different storage:

- Marks → small values
- Population → large values
- Count → always positive

Modifiers help:

- Save memory
- Increase range
- Improve efficiency

Common Type Modifiers

- **short** → smaller range
- **long** → larger range
- **unsigned** → only positive values
- **signed** → both positive and negative values

Type Modifiers with Different Data Types

Modifiers are mainly used with **integer and character types**.

Examples:

```
short int a = 10;
long int population = 2000000000;
unsigned int marks = 100;
signed int temp = -50;

unsigned char ch = 255;
```

Lesson Program: Type Modifiers and Type Casting in C++

Source Code: TypeModifying&Casting.cpp

```cpp
#include <iostream>
using namespace std;

int main() {

    // -------- Type Casting --------

    int a = 10, b = 3;

    // Without type casting
    float result1 = a / b;
    cout << "Without Type Casting: " << result1 << endl;

    // With type casting
    float result2 = (float)a / b;
    cout << "With Type Casting: " << result2 << endl;

    // Explicit type casting
    float x = 5.7;
    int y = (int)x;
    cout << "Float to Int: " << y << endl;

    // -------- Type Modifiers --------

    short int smallNum = 100;
    long int bigNum = 2000000000;

    unsigned int positiveNum = 40000;
    signed int negativeNum = -200;

    unsigned char ch = 250;

    // Displaying modified types
    cout << "Short int: " << smallNum << endl;
    cout << "Long int: " << bigNum << endl;
    cout << "Unsigned int: " << positiveNum << endl;
    cout << "Signed int: " << negativeNum << endl;
    cout << "Unsigned char: " << (int)ch << endl;

    return 0;
}
```

Operators in C++

In C++, operators are symbols that tell the computer to perform operations on data. Whether it is adding two numbers, comparing values, or assigning a result, operators are what make programs *do work* instead of just holding values. In simple terms: **Operators are used to perform operations on variables and constants**. For example: `int sum = a + b;` Here, + is an operator that adds two values.

Types of Operators in C++

C++ provides different types of operators based on the kind of operation they perform.

1. Arithmetic Operators
   - Perform basic mathematical operations.
   - Operators: + (addition), - (subtraction), * (multiplication), / (division), % (modulus).
   - Example (Math use):
     - 10 + 5 = 15
     - 10 - 5 = 5
     - 10 * 5 = 50
     - 10 / 5 = 2
     - 10 % 3 = 1
2. Relational Operators
   - Compare two values and return true (1) or false (0).
   - Operators: <, >, <=, >=, ==, !=.
   - Example (Math use):
     - 10 > 5 → true
     - 7 == 3 → false
3. Logical Operators
   - Used to combine relational expressions.
   - Operators: && (logical AND), || (logical OR), ! (logical NOT).
   - Example (Math use):
     - (10 > 5) && (5 < 20) → true
     - (10 < 5) || (5 < 20) → true
     - !(10 > 5) → false
4. Bitwise Operators
   - Perform operations on binary representations of integers.
   - Operators: & (AND), | (OR), ^ (XOR), ~ (NOT), << (left shift), >> (right shift).
   - Example (Binary use):
     - 5 & 3 → 1 (0101 & 0011 = 0001)
     - 5 | 3 → 7 (0101 | 0011 = 0111)
5. Assignment Operators
   - Assign values to variables.
   - Operators: =, +=, -=, *=, /=, %=.

- Example (Math use):
  - a = 10
  - a += 5 → a = a + 5

6. Unary Operators
   - Operate on a single operand.
   - Operators: ++ (increment), -- (decrement), ! (logical NOT), ~ (bitwise NOT).
   - Example (Math use):
     - a = 5, ++a → 6
     - a = 5, --a → 4
7. Ternary (Conditional) Operator
   - Shorthand for if-else condition.
   - Operator: ?:
   - Example (Math use):
     - (10 > 5) ? 1 : 0 → 1
8. Miscellaneous Operators
   - sizeof() → gives the size of a data type.
   - , (comma) → separate expressions.
   - & → address of variable.
   - * → pointer to variable.

Operator Precedence and Associativity:

When an expression contains multiple operators, C++ follows certain rules to decide which operation to perform first.

Precedence	Operator	Description	Associativity
1	() [] . → ++ --	Parentheses or function call Brackets or array subscript Dot or Member selection operator Arrow operator Postfix increment/decrement	left to right
2	++ – + - ! ~ (type) * & sizeof	Prefix increment/decrement Unary plus and minus NOT operator and bitwise complement Type case Indirection or dereference operator Address of operator Determine size in bytes	right to left
3	* / %	Multiplication, division, and modulus	left to right
4	+ -	Addition and subtraction	left to right
5	<< >>	Bitwise left shift and right shift	left to right
6	< <= > >=	Relational less than and less than or equal to Relational greater than and greater than or equal to	left to right
7	== !=	Relational equal to and not equal to	left to right
8	&	Bitwise AND	left to right
9	^	Bitwise exclusive OR	left to right
10	|	Bitwise inclusive OR	left to right
11	&&	Logical AND	left to right
12	||	Logical OR	left to right
13	?:	Ternary operator	right to left
14	= += -= *= /= %= &= ^= |= <<= >>=	Assignment operator Addition/ subtraction assignment Multiplication/ division assignment Modulus and bitwise assignment Bitwise exclusive and inclusive OR Bitwise left shift and right shift	right to left
15	,	Comma operator	left to right

Lesson Program: Operators in C++

Source Code: Operators.cpp

```cpp
#include <iostream>
using namespace std;

int main() {

    int a = 10, b = 5;

    // -------- Arithmetic Operators --------
    cout << "Arithmetic Operators:\n";
    cout << "a + b = " << a + b << endl;
    cout << "a - b = " << a - b << endl;
    cout << "a * b = " << a * b << endl;
    cout << "a / b = " << a / b << endl;
    cout << "a % b = " << a % b << endl;

    // -------- Relational Operators --------
    cout << "\nRelational Operators:\n";
    cout << "a > b: " << (a > b) << endl;
    cout << "a < b: " << (a < b) << endl;
    cout << "a == b: " << (a == b) << endl;
    cout << "a != b: " << (a != b) << endl;

    // -------- Logical Operators --------
    cout << "\nLogical Operators:\n";
    cout << "(a > 5 && b < 10): " << (a > 5 && b < 10) << endl;
    cout << "(a < 5 || b < 10): " << (a < 5 || b < 10) << endl;
    cout << "!(a > b): " << !(a > b) << endl;

    // -------- Assignment Operators --------
    cout << "\nAssignment Operators:\n";
    int x = 10;
    x += 5;
    cout << "x += 5: " << x << endl;

    x -= 3;
    cout << "x -= 3: " << x << endl;

    x *= 2;
    cout << "x *= 2: " << x << endl;

    x /= 4;
    cout << "x /= 4: " << x << endl;

    // -------- Increment / Decrement --------
    cout << "\nIncrement / Decrement:\n";
    int y = 5;
    cout << "y = " << y << endl;
    cout << "y++ = " << y++ << endl;  // post-increment
    cout << "After y++: " << y << endl;

    cout << "++y = " << ++y << endl;  // pre-increment
    cout << "After ++y: " << y << endl;

    // -------- Bitwise Operators --------
    cout << "\nBitwise Operators:\n";
    cout << "a & b = " << (a & b) << endl;
    cout << "a | b = " << (a | b) << endl;
    cout << "a ^ b = " << (a ^ b) << endl;
    cout << "~a = " << (~a) << endl;
    cout << "a << 1 = " << (a << 1) << endl;
    cout << "a >> 1 = " << (a >> 1) << endl;

    // -------- Conditional (Ternary) Operator --------
    cout << "\nTernary Operator:\n";
    int max = (a > b) ? a : b;
    cout << "Max of a and b: " << max << endl;

    // -------- sizeof Operator --------
    cout << "\nSizeof Operator:\n";
    cout << "Size of int: " << sizeof(a) << " bytes" << endl;
    cout << "Size of float: " << sizeof(float) << " bytes" << endl;

    // -------- Comma Operator --------
    cout << "\nComma Operator:\n";
    int m;
    m = (a = 3, b = 4, a + b);
    cout << "Result using comma operator: " << m << endl;

    // -------- Address-of Operator --------
    cout << "\nAddress-of Operator:\n";
    cout << "Address of a: " << &a << endl;

    return 0;
}
```

Input and Output in C++

A program becomes truly useful when it can **communicate** take input from the user and display output back. In C++, this communication is handled using **Input/Output (I/O)**. Instead of working silently, a program can ask questions, accept values, process them, and show results. This interaction is made possible through the **iostream library**.

What is iostream in C++?

iostream stands for Input Output Stream. It is a standard library in C++ that provides tools to:

- Take input from the user
- Display output on the screen

When you write: #include <iostream>
You are telling the compiler: "I want to use input and output features in my program". Without including iostream, you cannot use cin and cout.

What is a Stream?

A stream is a flow of data.

Think of it like a pipe 🚰:

- Input stream → data flows into the program
- Output stream → data flows out of the program

C++ uses streams to handle all input and output operations.

cout (Output Stream):

cout stands for **console output**. It is used to display output on the screen. "Send this data to the screen"

Example: cout << "Hello";

Here:

- << is called the **insertion operator**
- It sends data to the output stream

When to Use cout?

- To display messages
- To show results
- To print values of variables

cin (Input Stream):

cin stands for **console input**. It is used to take input from the user.

Example: cin >> age;

Here:

- >> is called the **extraction operator**
- It takes input from the user and stores it in a variable

"Take input and store it in this variable"

When to Use cin?

- When user input is required
- When values are not fixed
- When making programs interactive

Basic Input/Output Flow

1. Program asks for input (cout)
2. User enters data (cin)
3. Program processes data
4. Program displays result (cout)

Common Errors in Input/Output

Beginners often face small but important mistakes:

1. Missing #include <iostream>

   Error: cin or cout not recognized

2. Missing using namespace std;

   Error: Need to write std::cin, std::cout

3. Using wrong operator

   ```
   cin << age;   // ❌ Wrong
   cout >> age;  // ❌ Wrong
   ```

   Correct: cin >> age; cout << age;

4. Input Type Mismatch

   ```
   int age;
   cin >> age;   // user enters text → error
   ```

   Input must match variable type.

Think of cin and cout as basic conversation tools. The following features are like adding tone, formatting, and flexibility to that conversation.

1. getline() **Taking Full Line Input**

By default, cin reads input only **until the first space**. This becomes a problem when you want to read full names or sentences. It reads the entire line including spaces

Example: `string name; cin >> name;  // Input: John Doe → stores only "John"`
To solve this, C++ provides getline(): `getline(cin, name);`

When to Use getline()?

- Full names
- Sentences
- Multi-word input

2. endl **vs** \n

Both are used to move to a new line, but they behave slightly differently.

- \n → just moves to next line, `cout << "Hello\n";`
- endl → moves to next line and flushes the output buffer, `cout << "World" << endl;`

For beginners, both are fine, but \n is slightly faster.

3. iomanip **Formatting Output**

C++ provides a library called <iomanip> for formatting output. Include it like:

```
#include <iomanip>
```

Common Formatting Functions:

1. setw() → Set Width

   ```
   cout << setw(10) << 100;
   ```

   Prints value with spacing
2. setprecision() → Decimal Precision

   ```
   cout << setprecision(3) << 3.14159;
   ```

   Controls number of digits
3. fixed → Fixed Decimal Format

   ```
   cout << fixed << setprecision(2) << 3.14159;
   ```

   Output: 3.14
4. cerr → Error Output

   cerr is used to display error messages.

   ```
   cerr << "Error: Invalid input";
   ```

   It is unbuffered and prints immediately
5. clog → Logging Output

   clog is used for logging messages.

   ```
   clog << "Log: Program started";
   ```

   Used for debugging or tracking program flow

Summary of Common I/O Tools

Tool	Purpose
cin	Input
cout	Output
getline()	Full line input
endl / \n	New line
setw()	Formatting width
setprecision()	Decimal control
cerr	Error messages
clog	Logging

Lesson Program: Input & Output in C++

Source Code: InputOutput.cpp

```cpp
#include <iostream>
#include <iomanip>
using namespace std;

int main() {

    // -------- Basic Input using cin --------
    string name;
    int age;
    float marks;

    cout << "Enter your name (single word): ";
    cin >> name;

    cout << "Enter your age: ";
    cin >> age;

    cout << "Enter your marks: ";
    cin >> marks;

    // -------- Displaying Output --------
    cout << "\n--- Student Details ---\n";
    cout << "Name: " << name << endl;
    cout << "Age: " << age << endl;
    cout << "Marks: " << marks << endl;

    // -------- Calculation --------
    int nextAge = age + 1;
    cout << "Next year, you will be: " << nextAge << endl;

    // -------- getline() Example --------
    string fullName;

    cin.ignore(); // clear buffer before getline

    cout << "\nEnter your full name: ";
    getline(cin, fullName);

    cout << "Full Name: " << fullName << endl;

    // -------- Formatting Output --------
    float value = 3.14159;

    cout << "\nWithout formatting: " << value << endl;

    cout << "Using setprecision(3): "
         << setprecision(3) << value << endl;

    cout << "Using fixed and setprecision(2): "
         << fixed << setprecision(2) << value << endl;

    // -------- setw() Example --------
    cout << "\nUsing setw():\n";
    cout << setw(10) << 100 << endl;
    cout << setw(10) << 200 << endl;

    // -------- cerr Example --------
    cerr << "\nError: This is a sample error message\n";

    // -------- clog Example --------
    clog << "Log: Program executed successfully\n";

    return 0;
}
```

// Other Input & Output Methods in C++

So far, we have used cin and cout from the <iostream> library for input and output. While they are the most common, C++ also provides **other ways to take input and display output**, especially through functions inherited from C and some additional utilities.

These methods are useful when:

- You need simpler or faster input/output
- You want character-based or formatted input
- You are working with legacy code or competitive programming

1. scanf() and printf() (from C)

C++ also supports C-style input/output using the <cstdio> library. They are formatted I/O functions, meaning you must specify the type of data.
Include it like: #include <cstdio>

What are scanf() and printf()?

- scanf() → used for input
- printf() → used for output

Example

```
int age;
scanf("%d", &age);
printf("Age = %d", age);
```

When to Use?

- Faster input/output (used in competitive programming)
- When working with C-style code

2. getchar() and putchar()

These are used for **single character input and output**.

What are they?

- getchar() → reads a single character
- putchar() → prints a single character

Example:

```
char ch;
ch = getchar();
putchar(ch);
```

When to Use?

- Reading character-by-character input
- Simple input/output tasks

3. gets() and puts() (Basic Idea):

These are used for string input/output (from C-style).

- gets() → reads a string
- puts() → prints a string

Note: gets() is **unsafe and not recommended** in modern C++.

Lesson Program: Input & Output other Methods in C++

Source Code: OtherI&O.cpp

```cpp
#include <iostream>
#include <cstdio>
using namespace std;

int main() {

    // -------- scanf and printf --------
    int age;
    float marks;

    printf("Enter age: ");
    scanf("%d", &age);

    printf("Enter marks: ");
    scanf("%f", &marks);

    printf("Age = %d\n", age);
    printf("Marks = %.2f\n", marks);

    // -------- getchar and putchar --------
    char ch;

    printf("\nEnter a character: ");
    getchar(); // to consume leftover newline
    ch = getchar();

    printf("You entered: ");
    putchar(ch);
    printf("\n");

    // -------- puts --------
    puts("\nThis is a string using puts");

    return 0;
}
```

Program Flow in C++

When a C++ program runs, the computer follows a specific order to execute instructions. This order is called the **program flow** or **control flow** of the program.

By default, C++ executes statements **one after another from top to bottom**. The computer starts from the first executable statement and continues step by step until the program ends.

Here, the execution happens in a straight line:

1. First statement executes
2. Then second
3. Then third

This is called **sequential execution**.

Why Is Program Flow Important?

Real programs are rarely simple straight-line instructions. Sometimes a program needs to: make decisions, repeat actions, skip certain instructions, stop execution early, jump to another part of the program

Imagine a login system:

- If password is correct → allow access
- Otherwise → show error

Or think about a game:

- Repeat actions until the player loses
- Increase score continuously

Programs need flexibility. They cannot always move only in one direction like a train on a single track. This is where control flow statements become important.

What Are Control Flow Statements?

Control flow statements are special statements that change the normal execution flow of a program. They allow the program to:

- Choose between different paths
- Repeat blocks of code
- Interrupt execution
- Transfer control to another statement

Without control flow statements, programs would only execute instructions in fixed order and would never be able to make decisions or perform repetition.

Types of Program Flow in C++

Program execution in C++ mainly follows three kinds of flow:

1. Sequential Flow

This is the default flow. Statements execute one after another in the order they are written. Each line executes sequentially.

2. Selection (Decision) Flow

Sometimes the program needs to decide which path to follow. Example: If marks are above 40 → pass, Else → fail

This is done using:

- if
- if-else
- switch

3. Repetition (Looping) Flow

Sometimes a task must repeat multiple times. Example: Print numbers from 1 to 10, Keep asking for password until correct

This is done using:

- for
- while
- do-while

Flow Disruption in Programs

Control flow statements disrupt the normal top-to-bottom execution.

For example:

- Loops send execution backward repeatedly 🔁
- Conditions choose one path among many 🔀
- Jump statements like break or continue interrupt flow instantly ⚡

Because of this, program execution becomes dynamic instead of fixed.

Understanding Program Flow Visually

You can imagine program flow like water flowing through pipes 🚰

- Normal execution → water flows straight
- Conditions → water chooses a branch
- Loops → water circulates repeatedly
- Jump statements → water suddenly changes direction

The programmer controls this flow using control statements.

Conditional Statements in C++

Conditional statements (also called decision-making statements) are used in C++ to make programs choose different paths of execution depending on whether a condition is true or false.
In real life, decisions happen constantly:

- If it rains → take an umbrella
- If marks are above 40 → pass
- If password is correct → allow login

Programs work the same way. They evaluate conditions and decide what should happen next. Conditional statements make programs more dynamic, intelligent, and responsive to different situations.

Types of Conditional Statements in C++

1. if Statement

The if statement is used to execute a block of code only when a condition is true.

Syntax:

```
if (condition) {
    // code to execute if condition is true
}
```

Example:

```
if (age >= 18) {
    cout << "You can vote";
}
```

2. if-else Statement

The if-else statement provides two-way decision making.

- If the condition is true → if block executes
- Otherwise → else block executes

Syntax:

```
if (condition) {
    // code if condition is true
} else {
    // code if condition is false
}
```

Example:

```
if (num % 2 == 0) {
    cout << "Even";
} else {
    cout << "Odd";
}
```

3. else-if Ladder

Used when multiple conditions need to be checked. The first true condition executes, and the remaining conditions are skipped.

Syntax:

```
if (condition1) {
   // code
} else if (condition2) {
   // code
} else {
   // code if none are true
}
```

Example:

```
if (marks >= 90) {
   cout << "Grade A";
} else if (marks >= 75) {
   cout << "Grade B";
} else {
   cout << "Grade C or Fail";
}
```

4. Nested if Statement

A nested if means an if statement inside another if statement.It is used for multiple levels of checking.

Syntax

```
if (condition1) {
   if (condition2) {
      // code if both are true
   }
}
```

Example

```
if (num > 0) {
   if (num % 2 == 0) {
      cout << "Positive Even Number";
   }
}
```

5. switch Statement

The switch statement is used when there are multiple fixed choices.

It works with:

- Integer values
- Character values
- Constant expressions

Instead of checking many conditions using else-if, switch directly jumps to the matching case.

Syntax

```
switch (expression) {
  case value1:
    // code
    break;
  case value2:
    // code
    break;
  default:
    // code if no case matches
}
```

Example

```
switch (day) {
  case 1:
    cout << "Monday";
    break;
  case 2:
    cout << "Tuesday";
    break;
  case 3:
    cout << "Wednesday";
    break;
  default:
    cout << "Invalid Day";
}
```

Rules for switch Statement in C++

1. The switch expression must evaluate to:
   - int
   - char
   - enum
2. Case values must be unique constants.
3. break is optional but recommended.
   - With break → exits switch block
   - Without break → execution continues to next case (fall-through)
4. default executes if no case matches.

Closing Thought

Conditional statements give programs the ability to make decisions. Without them, programs would blindly execute every instruction in order. With them, programs can react, choose paths, and behave differently depending on situations. They are the beginning of real program logic.

Lesson Program: if & if-else statements in C++
Source Code: ConditionalStatements1.cpp

```cpp
#include <iostream>
using namespace std;

int main() {

    int age, num;

    // if statement example:
    cout << "Enter your age: ";
    cin >> age;

    // Checking condition
    if (age >= 18) {
        cout << "You are eligible to vote.";
    }

    // if-else statement example:
    cout << "Enter a number: ";
    cin >> num;

    // Checking even or odd
    if (num % 2 == 0) {
        cout << "The number is Even.";
    } else {
        cout << "The number is Odd.";
    }

    return 0;
}
```

Lesson Program: else-if ladder & nested if-else statements in C++
Source Code: ConditionalStatements2.cpp

```cpp
#include <iostream>
using namespace std;

int main() {

    int marks, num;

    // else-if ladder statement example:
    cout << "Enter your marks: ";
    cin >> marks;

    // Checking grades
    if (marks >= 90) {
        cout << "Grade A";
    } else if (marks >= 75) {
        cout << "Grade B";
    } else if (marks >= 50) {
        cout << "Grade C";
    } else {
        cout << "Fail";
    }

    // Nested if-else statement example:
    cout << "Enter a number: ";
    cin >> num;

    // Outer if
    if (num > 0) {
        // Inner if
        if (num % 2 == 0) {
            cout << "Positive Even Number";
        } else {
            cout << "Positive Odd Number";
        }
    } else {
        cout << "Number is Zero or Negative";
    }

    return 0;
}
```

Lesson Program: switch statements in C++
Source Code: ConditionalStatements3.cpp

```cpp
#include <iostream>
using namespace std;

int main() {

    int day;

    cout << "Enter day number (1-7): ";
    cin >> day;

    // Switch statement example:
    switch (day) {
        case 1:
            cout << "Monday";
            break;
        case 2:
            cout << "Tuesday";
            break;
        case 3:
            cout << "Wednesday";
            break;
        case 4:
            cout << "Thursday";
            break;
        case 5:
            cout << "Friday";
            break;
        case 6:
            cout << "Saturday";
            break;
        case 7:
            cout << "Sunday";
            break;
        default:
            cout << "Invalid Day";
    }

    return 0;
}
```

Looping Statements in C++

Sometimes a program needs to execute the same block of code multiple times. Writing the same statements again and again would make programs lengthy and inefficient. To solve this, C++ provides **looping statements**.

Loops allow a program to repeat instructions until a condition becomes false. In real life, repetition happens everywhere:

- Printing numbers from 1 to 100
- Sending reminders every day
- Repeating game actions until the player loses

Loops make programs faster, cleaner, and more dynamic.

Types of Loops in C++

1. for Loop

The for loop is used when the number of repetitions is known beforehand. It combines: Initialization, Condition, Update all in a single line.

Syntax:

```
for (initialization; condition; update) {
   // code to repeat
}
```

Example:

```
for (int i = 1; i <= 5; i++) {
   cout << i << endl;
}
```

This prints numbers from 1 to 5.

2. while Loop

The while loop executes as long as the condition remains true.
It is mainly used when the number of repetitions is not fixed.

Syntax

```
while (condition) {
   // code to repeat
}
```

Example

```
int i = 1;
while (i <= 5) {
   cout << i << endl;
   i++;
}
```

3. do-while Loop

The do-while loop is similar to while, but with one important difference: The loop body executes at least once, even if the condition is false.

This happens because the condition is checked **after** execution.

Syntax:

```
do {
   // code to repeat
} while (condition);
```

Example:

```
int i = 1;
do {
   cout << i << endl;
   i++;
} while (i <= 5);
```

4. Nested Loops

A loop inside another loop is called a nested loop.

Nested loops are commonly used for:

- Patterns
- Tables
- Matrix operations

Syntax:

```
for (...) {
   for (...) {
      // inner loop code
   }
}
```

Example:

```
for (int i = 1; i <= 3; i++) {
   for (int j = 1; j <= 2; j++) {
      cout << "* ";
   }
   cout << endl;
}
```

Output:

```
* *
* *
* *
```

Common Rules & Errors in Loops

Rules:

1. Always make sure the condition will eventually become false, otherwise → infinite loop.
2. Any of the three parts in for(initialization; condition; update) can be omitted, but semicolons must remain.
    - Example: for(; ;) → infinite loop.
3. Variables declared in initialization are local to the loop.

Common Errors

- Missing semicolons in for or do-while.

```
do {
        ...
} while (condition)   // ❌ error, needs ;
```

- Wrong condition leading to infinite loop.
- Forgetting update statement, causing infinite loop.
- Placing a semicolon immediately after loop (empty loop body).

```
for (i=0; i<5; i++);  // ❌ does nothing
```

Feature	for loop	while loop	do-while loop
Condition checked	Before	Before	After
Executed at least once?	❌ No	❌ No	☑ Yes
Use case	Known iterations	Unknown iterations	Must run once

Closing Thought

Loops are what give programs rhythm 🔁 Without loops, programs would repeat code endlessly by hand. With loops, a few lines of code can perform thousands of operations automatically. They are one of the most powerful tools in programming because they allow repetition, automation, and efficiency.

Lesson Program: for loop statements in C++
Source Code: LoopingStatements1.cpp

```cpp
#include <iostream>
using namespace std;

int main() {

    // for loop example 1:
    // Printing numbers from 1 to 5
    cout << "Numbers from 1 to 5:\n";

    for (int i = 1; i <= 5; i++) {
        cout << i << endl;
    }

    // for loop example 2:
    // Printing even numbers
    cout << "\nEven numbers from 2 to 10:\n";

    for (int i = 2; i <= 10; i += 2) {
        cout << i << endl;
    }

    return 0;
}
```

Lesson Program: while loop statements in C++
Source Code: LoopingStatements2.cpp

```cpp
#include <iostream>
using namespace std;

int main() {

    // while loop example 1:
    // Printing numbers from 1 to 5
    int i = 1;

    cout << "Numbers from 1 to 5:\n";

    while (i <= 5) {
        cout << i << endl;
        i++;
    }

    // while loop example 2:
    // Countdown
    int count = 5;

    cout << "\nCountdown:\n";

    while (count >= 1) {
        cout << count << endl;
        count--;
    }

    return 0;
}
```

Lesson Program: do-while loop statements in C++
Source Code: LoopingStatements3.cpp

```cpp
#include <iostream>
using namespace std;

int main() {

    // do-while loop example 1:
    // Printing numbers from 1 to 5
    int i = 1;

    cout << "Numbers from 1 to 5:\n";

    do {
        cout << i << endl;
        i++;
    } while (i <= 5);

    // do-while loop example 2:
    // Printing multiplication table of 2
    int num = 1;

    cout << "\nTable of 2:\n";

    do {
        cout << "2 x " << num << " = " << 2 * num << endl;
        num++;
    } while (num <= 10);

    return 0;
}
```

Lesson Program: nested loop statements in C++
Source Code: LoopingStatements4.cpp

```cpp
#include <iostream>
using namespace std;

int main() {

    // nested loop example 1:
    // Printing square pattern
    cout << "Square Pattern:\n";

    for (int i = 1; i <= 3; i++) {
        for (int j = 1; j <= 3; j++) {
            cout << "* ";
        }
        cout << endl;
    }

    // nested loop example 2:
    // Multiplication table
    cout << "\nMultiplication Table:\n";

    for (int i = 1; i <= 3; i++) {
        for (int j = 1; j <= 3; j++) {
            cout << i * j << "\t";
        }
        cout << endl;
    }

    return 0;
}
```

Jump Statements in C++

Jump statements in C++ are used to alter the normal flow of program execution. They allow the control of the program to move immediately from one part of the program to another.

Normally, programs execute sequentially from top to bottom. But sometimes we need to:

- Stop a loop early
- Skip certain iterations
- Jump directly to another part of the program
- Exit a function immediately

Jump statements provide this control.

Types of Jump Statements in C++

1. break Statement

The break statement is used to immediately terminate a loop or switch statement. Once break executes: Control moves outside the loop or switch block.

Where is break Used?

- for loop
- while loop
- do-while loop
- switch statement

Syntax: break;

2. continue Statement

The continue statement skips the current iteration of a loop and moves directly to the next iteration. Unlike break, it does not terminate the loop completely.

Where is continue Used?

- for loop
- while loop
- do-while loop

Syntax: continue;

3. goto Statement

The goto statement transfers control directly to a labeled statement inside the same function. It works like a jump in execution flow.

Syntax:

```
goto label;
...
label:
   statement;
```

Basic Idea of goto:

goto can make execution jump forward or backward in a program. Although it exists in C++, excessive use of goto is discouraged because it can make programs difficult to read and debug. This is often called **spaghetti code** 🍝

Modern programming usually prefers loops and functions instead of goto.

4. return Statement

The return statement is used to terminate a function and optionally send a value back. In main(), return 0; usually indicates that the program ended successfully.

Once return executes: The function immediately stops execution.

Syntax: return value; or return;

Closing Thought

Jump statements are like emergency doors and shortcuts inside a program:

- break exits
- continue skips
- goto jumps
- return ends execution

They give precise control over how a program moves and behaves. The real skill is not just knowing them, but knowing when a jump improves clarity and when it creates confusion.

Lesson Program: Jump statements in C++
Source Code: JumpStatements.cpp

```cpp
#include <iostream>
using namespace std;

int main() {

    // -------- break Statement --------
    cout << "Break Statement:\n";

    for (int i = 1; i <= 10; i++) {
        if (i == 5) {
            break;
        }
        cout << i << " ";
    }

    // -------- continue Statement --------
    cout << "\n\nContinue Statement:\n";

    for (int i = 1; i <= 5; i++) {
        if (i == 3) {
            continue;
        }
        cout << i << " ";
    }

    // -------- goto Statement --------
    cout << "\n\nGoto Statement:\n";

    goto message;

    cout << "This line will be skipped\n";
    cout << "This line will be skipped\n";
    cout << "This line will be skipped\n";

message:
    cout << "Control jumped using goto";

    // -------- return Statement --------
    cout << "\n\nReturn Statement:\n";

    int num = -5;

    if (num < 0) {
        cout << "Negative number detected. Program ending...\n";
        return 0;
    }

    cout << "This line will not execute";

    return 0;
}
```

Arrays in C++

Until now, we have used variables to store single values.
Example: int age = 21;
But what if a program needs to store:

- Marks of 100 students
- Temperatures of 30 days
- Scores of multiple players

Creating separate variables for each value would become difficult and messy. This is where **arrays** become useful.

What is an Array?

An array is a collection of multiple values of the **same data type** stored under a single name. Instead of creating many variables, an array allows us to store related data together.
👉 An array is like a row of storage boxes 📦 📦 📦 Each box stores one value, and every box has a position number called an **index**.

Why Do We Need Arrays?

Arrays help:

- Store large amounts of related data
- Reduce number of variables
- Process multiple values easily using loops
- Organize data efficiently

Without arrays: `int mark1, mark2, mark3, mark4;`
With arrays: `int marks[4];`
Cleaner, simpler, and easier to manage.

How Arrays Work in Memory

Array elements are stored in **continuous memory locations**.
Each element has:

- Same data type
- Fixed position (index)

Example: `int marks[5];`
Memory representation:

```
marks[0]
marks[1]
marks[2]
marks[3]
marks[4]
```

Array indexing always starts from 0.
So:

- First element → index 0
- Second element → index 1

and so on.

Declaring an Array

Declaration means creating an array.

Syntax: `dataType arrayName[size];`
Example: `int marks[5];`
This creates an integer array capable of storing 5 values.

Initializing an Array

Initialization means assigning values to the array.

Syntax: `dataType arrayName[size] = {values};`
Example: `int marks[5] = {90, 85, 78, 92, 88};`
Here:

- marks[0] = 90
- marks[1] = 85
- and so on

Accessing Array Elements

Array elements are accessed using their index number.

Syntax: `arrayName[index]`
Example: `cout << marks[2];`
Output: 78
Because: marks[2] = 78

Modifying Array Elements

Values inside arrays can also be changed.

Example: `marks[1] = 95;`
Now the second element becomes 95.

Accessing All Array Elements Using Loops

Loops are commonly used with arrays because arrays usually contain multiple values.
Instead of writing:

```
cout << marks[0];
cout << marks[1];
cout << marks[2];
```

we use loops to access elements automatically.

Using for Loop:

```
for (int i = 0; i < 5; i++) {
    cout << marks[i] << endl;
}
```

Here:

- i acts as the index number
- Loop starts from 0
- Continues until last index

This prints all array elements one by one.

Using while Loop:

```
int i = 0;
while (i < 5) {
    cout << marks[i] << endl;
    i++;
}
```

Notes to Remember While Using Arrays

1. Array Index Starts from 0
   For an array of size 5:
   0 1 2 3 4
2. Array Size is Fixed
   Once declared:
   int arr[5];
   Its size cannot be changed.
3. All Elements Must Be Same Type
   int arr[5];
   Can only store integers.
4. Accessing Invalid Index Causes Problems
   arr[10]
   This may cause unexpected behavior.
5. Loops are Commonly Used with Arrays
   Loops make arrays powerful and practical.

Closing Thought:

Arrays are the first step toward handling large collections of data efficiently. Instead of thinking in terms of single values, arrays teach you to think in groups, sequences, and patterns. Once arrays enter programming, your programs begin to scale from handling one thing to handling many.

Lesson Program: Arrays in C++

Source Code: Arrayss1.cpp

```cpp
#include <iostream>
using namespace std;

int main() {

    // Declaring an array
    int marks[5];

    // Initializing array elements
    marks[0] = 90;
    marks[1] = 85;
    marks[2] = 78;
    marks[3] = 92;
    marks[4] = 88;

    // Displaying array elements
    cout << "Marks stored in array:\n";

    for (int i = 0; i < 5; i++) {
        cout << "marks[" << i << "] = " << marks[i] << endl;
    }

    return 0;
}
```

Lesson Program: Arrays input & output in C++

Source Code: Arrayss2.cpp

```cpp
#include <iostream>
using namespace std;

int main() {

    int numbers[5];

    // Taking input into array
    cout << "Enter 5 numbers:\n";

    for (int i = 0; i < 5; i++) {
        cin >> numbers[i];
    }

    // Displaying array elements
    cout << "\nElements in the array are:\n";

    for (int i = 0; i < 5; i++) {
        cout << "numbers[" << i << "] = " << numbers[i] << endl;
    }

    return 0;
}
```

Lesson Program: Arrays of all data types in C++

Source Code: Arrayss3.cpp

```cpp
#include <iostream>
using namespace std;

int main() {

    int numbers[3] = {10, 20, 30};      // Integer array
    float prices[3] = {99.5, 150.75, 200.25};     // Float array
    char grades[3] = {'A', 'B', 'C'};     // Character array
    bool status[3] = {true, false, true};     // Boolean array
    string names[3] = {"Aditya", "Rahul", "Aman"};     // String array


    // Displaying integer array
    cout << "Integer Array:\n";

    for (int i = 0; i < 3; i++) {
        cout << numbers[i] << endl;
    }

    // Displaying float array
    cout << "\nFloat Array:\n";

    for (int i = 0; i < 3; i++) {
        cout << prices[i] << endl;
    }
```

```cpp
    // Displaying character array
    cout << "\nCharacter Array:\n";

    for (int i = 0; i < 3; i++) {
        cout << grades[i] << endl;
    }

    // Displaying boolean array
    cout << "\nBoolean Array:\n";

    for (int i = 0; i < 3; i++) {
        cout << status[i] << endl;
    }

    // Displaying string array
    cout << "\nString Array:\n";

    for (int i = 0; i < 3; i++) {
        cout << names[i] << endl;
    }

    return 0;
}
```

// Multidimensional Arrays

So far, we have worked with one-dimensional arrays, where data is stored in a single sequence. But in many real-world situations, data is organized in rows and columns instead of a simple line.

For example:

- Marks of students in different subjects
- Seating arrangement in a classroom
- Matrix calculations
- Game boards like chess or tic-tac-toe

To store and manage such structured data, C++ provides **multidimensional arrays**. A multidimensional array is simply an array that contains other arrays.

1D Arrays in C++

A one-dimensional array stores elements in a single row or sequence.

Example: `int numbers[5] = {10, 20, 30, 40, 50};`
Memory representation: `numbers[0]` `numbers[1]` `numbers[2]` `numbers[3]` `numbers[4]`
Only one index is needed to access elements.

Syntax of 1D Array: `dataType arrayName[size];`
Accessing Elements: `numbers[2]`
Output: 30

2D Arrays in C++

A two-dimensional array stores data in the form of **rows and columns**, similar to a table or matrix. You can think of it as: An array of arrays

Example: `int marks[2][3];`
This creates:

- 2 rows
- 3 columns

Memory representation:

```
marks[0][0]  marks[0][1]  marks[0][2]
marks[1][0]  marks[1][1]  marks[1][2]
```

Two indices are required:

- First index → row
- Second index → column

Declaring a 2D Array

Syntax: dataType arrayName[rows][columns];

Example: int matrix[2][3];

Initializing a 2D Array

Syntax:

```
dataType arrayName[rows][columns] = {
   {values},
   {values}
};
```

Example:

```
int matrix[2][3] = {
   {1, 2, 3},
   {4, 5, 6}
};
```

Accessing Elements in 2D Arrays

Syntax: arrayName[row][column]

Example: cout << matrix[1][2];

Output: 6

Because: matrix[1][2] = 6

Array Traversal

Traversal means visiting and processing every element of an array one by one. Loops are commonly used for traversal. Since 2D arrays contain rows and columns, nested loops are used.

Example:

```
for (int i = 0; i < 2; i++) {
   for (int j = 0; j < 3; j++) {
      cout << matrix[i][j] << " ";
   }
   cout << endl;
}
```

Here:

- Outer loop → rows
- Inner loop → columns

Notes to Remember

1. Indexing Starts from 0
   Both 1D and 2D arrays use zero-based indexing.
2. Arrays Store Same Data Type
   A single array cannot mix integers, floats, characters, etc.
3. Nested Loops are Important for 2D Arrays
   2D arrays are almost always used with nested loops.
4. Size Must Be Defined
   The compiler needs array size during declaration.

Closing Thought

Arrays organize data. Multidimensional arrays organize relationships between data. Once arrays move from lines into rows and columns, programs begin handling information in a more structured and realistic way, much closer to how data exists in the real world.

Lesson Program: Multidimensional Arrays in C++
Source Code: MultiDimArrayss1.cpp

```cpp
#include <iostream>
using namespace std;

int main() {

    // Initializing a 2D array
    int matrix[2][3] = {
        {1, 2, 3},
        {4, 5, 6}
    };

    // Displaying the 2D array
    cout << "Elements of 2D Array:\n";

    for (int i = 0; i < 2; i++) {
        for (int j = 0; j < 3; j++) {
            cout << matrix[i][j] << " ";
        }
        cout << endl;
    }

    return 0;
}
```

Lesson Program: Multidimensional Arrays in C++

Source Code: MultiDimArrays2.cpp

```cpp
#include <iostream>
using namespace std;

int main() {

    // Initializing array
    int marks[3][3] = {
        {90, 85, 88},
        {78, 92, 80},
        {95, 89, 91}
    };

    // Accessing individual elements
    cout << "Accessing Individual Elements:\n";

    cout << "marks[0][0] = " << marks[0][0] << endl;
    cout << "marks[1][2] = " << marks[1][2] << endl;
    cout << "marks[2][1] = " << marks[2][1] << endl;

    return 0;
}
```

Lesson Program: Multidimensional Arrays in C++

Source Code: MultiDimArrays3.cpp

```cpp
#include <iostream>
using namespace std;

int main() {

    int matrix[2][3];

    // Taking input
    cout << "Enter elements for 2D array:\n";

    for (int i = 0; i < 2; i++) {
        for (int j = 0; j < 3; j++) {
            cin >> matrix[i][j];
        }
    }

    // Displaying output
    cout << "\nElements of the 2D Array:\n";

    for (int i = 0; i < 2; i++) {
        for (int j = 0; j < 3; j++) {
            cout << matrix[i][j] << " ";
        }
        cout << endl;
    }

    return 0;
}
```

Strings in C++

Until now, we have worked with numbers, characters, and arrays. But many real-world programs need to handle text.

For example:

- A student's name
- A city name
- A password
- A sentence entered by a user

Text data is represented using **strings**.

Strings are one of the most commonly used data types in programming because programs often need to store, process, compare, and display textual information.

What is a String?

A string is a sequence of characters stored together as a single unit.

Examples:

"Hello"

"Aditya"

"C++ Programming"

Each string consists of multiple characters arranged in order.

For example: "Hello"

contains: H e l l o

Why Do We Need Strings?

Suppose you want to store a person's name.

Without strings:

```
char c1 = 'A';
char c2 = 'd';
char c3 = 'i';
```

This quickly becomes impractical.

Instead: `string name = "Aditya";`

Strings allow us to store entire words, names, and sentences easily.

How Strings are Stored

A string is actually a collection of characters stored one after another in memory.
Example: "Hello"
Memory representation: H e l l o
Each character occupies its own memory location. This relationship between strings and characters is important because it explains why strings can be accessed character by character.

Strings in C++

C++ supports two major ways of working with strings:

1. Character Arrays (C-Style Strings)
2. String Class (std::string)

1. Character Arrays (C-Style Strings)

Before the C++ string class existed, strings were represented using arrays of characters.
Example: `char name[] = "Aditya";`
This creates a character array containing: A d i t y a \0

Null Character (\0): Every C-style string ends with a special character called the **null character**.'\0'

It tells the computer where the string ends. Without it, the program would not know where to stop reading characters.
Example: `char word[] = "Hello";`
Stored as: H e l l o \0

Advantages of Character Arrays

- Memory efficient
- Compatible with C programs
- Useful for low-level programming

Limitations of Character Arrays

- Fixed size
- Manual handling required
- String operations are more difficult

Example: `char name[20];`
The size must be specified in advance.

2. String Class (std::string)

Modern C++ provides the **string class**, which is easier and safer to use.
The string class belongs to the: <string> library.

Including String Library

```
#include <string>
```

Although many compilers include it automatically through <iostream>, it is good practice to include it explicitly.

Creating a String

```
string name = "Aditya";
```

or

```
string city;
```

Advantages of String Class

- Dynamic size
- Easy input/output
- Built-in functions
- Safer than character arrays
- Easier to manipulate text

Example:

```
string name = "Aditya";
    cout << name;
```

Output: Aditya

Input and Output with Strings

Using cin:

```
string name;
cin >> name;
```

Input: Aditya

Limitation of cin: cin stops reading at the first space.
Input: Aditya Varma
Stored: Aditya
Only the first word is stored.

Using getline():

To read complete lines:

```
string name;
getline(cin, name);
```

Input: Aditya Varma
Stored: Aditya Varma

Accessing Characters in a String

Individual characters can be accessed using indexes.

Example:

```
string word = "Hello";
cout << word[0];
```

Output: H

Length of a String

The string class can determine the number of characters stored.

```
string word = "Hello";
cout << word.length();
```

Output: 5

Comparing Character Arrays and String Class

Character Array	String Class
Uses char[]	Uses string
Fixed size	Dynamic size
Manual operations	Built-in functions
More complex	Easier to use
Older C-style approach	Modern C++ approach

When to Use Which?

Use Character Arrays When:

- Learning how strings work internally
- Working with legacy C code
- Memory optimization is critical

Use String Class When:

- Writing modern C++ programs
- Handling user input
- Working with text data

Most modern C++ programs use the **string class**.

Notes to Remember

1. Strings are collections of characters.
2. Character arrays end with '\0'.
3. String class belongs to the <string> library.
4. cin reads only one word.
5. getline() reads entire lines.

6. String indexing starts from 0.
Example: string name = "CPLUSPLUS";
Indexes: 0 1 2 3 4 5 6 7 8
C P L U S P L U S

Closing Thought

Strings allow programs to work with human language. While numbers help a program calculate, strings help it communicate. Understanding both C-style strings and the modern string class gives you a complete picture of how text is stored, processed, and manipulated in C++.

Lesson Program: Strings in C++
Source Code: Strings1.cpp

```cpp
#include <iostream>
#include <string>
using namespace std;

int main() {

    // Declaring strings
    string firstName;
    string lastName;

    // Initializing strings
    firstName = "John";
    lastName = "Doe";

    // Displaying strings
    cout << "First Name: " << firstName << endl;
    cout << "Last Name: " << lastName << endl;

    // Direct initialization
    string course = "C++ Programming";

    cout << "Course: " << course << endl;

    return 0;
}
```

Lesson Program: Strings indexing in C++

Source Code: Strings2.cpp

```cpp
#include <iostream>
#include <string>
using namespace std;

int main() {

    // Initializing string
    string word = "PROGRAMMING";

    // Accessing individual characters
    cout << "First Character: " << word[0] << endl;
    cout << "Third Character: " << word[2] << endl;
    cout << "Last Character: "  << word[word.length() - 1] << endl;

    // Accessing all characters using loop
    cout << "\nAll Characters:\n";

    for (int i = 0; i < word.length(); i++) {
        cout << word[i] << " ";
    }

    return 0;
}
```

Lesson Program: Strings input & output

Source Code: Strings3.cpp

```cpp
#include <iostream>
#include <string>
using namespace std;

int main() {
    string fullName;
    string city;

    // Taking string input
    cout << "Enter your full name: ";
    getline(cin, fullName);

    cout << "Enter your city: ";
    getline(cin, city);

    // Displaying information
    cout << "\n--- Student Information ---\n";

    cout << "Name: " << fullName << endl;
    cout << "City: " << city << endl;

    return 0;
}
```

Lesson Program: Strings Functions in C++

Source Code: Strings4.cpp

```cpp
#include <iostream>
#include <string>
using namespace std;

int main() {
    string firstName = "Aditya";
    string lastName = "Varma";

    // Concatenation
    string fullName = firstName + " " + lastName;

    // Displaying strings
    cout << "First Name: " << firstName << endl;
    cout << "Last Name: " << lastName << endl;
    cout << "Full Name: " << fullName << endl;

    // Length/Size
    cout << "\nLength: " << fullName.length() << endl;
    cout << "Size: " << fullName.size() << endl;

    // Accessing characters
    cout << "\nFirst Character: " << fullName.front() << endl;
    cout << "Last Character: " << fullName.back() << endl;

    // Substring
    cout << "\nSubstring (first 6 characters): " << fullName.substr(0, 6) << endl;

    // Finding text
    cout << "\nPosition of 'Varma': " << fullName.find("Varma") << endl;

    // Appending text
    fullName.append(" Kumar");
    cout << "\nAfter append(): " << fullName << endl;

    // Inserting text
    fullName.insert(7, "Mr. ");
    cout << "After insert(): " << fullName << endl;

    // Erasing text
    fullName.erase(7, 4);
    cout << "After erase(): " << fullName << endl;

    // Replacing text
    fullName.replace(0, 6, "Aman");
    cout << "After replace(): " << fullName << endl;

    // Empty check
    cout << "\nIs string empty? " << fullName.empty() << endl;

    // Clearing string
    string temp = "Hello";
    cout << "\nBefore clear(): " << temp << endl;

    temp.clear();

    cout << "After clear(): " << temp << endl;
    cout << "Is temp empty? " << temp.empty() << endl;

    return 0;
}
```

Pointers in C++

Until now, we have worked with variables that store values directly.
Example: `int age = 23;`
Here, the variable age stores the value 23.

But every variable is stored somewhere in the computer's memory. Along with storing values, C++ also allows us to work with the **memory addresses** where those values are stored.

This is where **pointers** come into the picture.

Pointers are one of the most powerful features of C++. They provide direct access to memory and form the foundation for advanced topics such as dynamic memory allocation, data structures, and object-oriented programming.

What is a Pointer?

A pointer is a special variable that stores the **memory address** of another variable. Unlike normal variables that store values, pointers store locations.

For example: `int age = 23;`
Suppose age is stored at memory address: 1000
A pointer can store this address: `int *ptr = &age;`
Now:

```
age = 23
ptr = 1000
```

Here:

- age stores the value
- ptr stores the address of age

Why Do We Need Pointers?

Pointers allow programs to:

- Access memory directly
- Share data between functions efficiently
- Work with arrays and strings
- Create dynamic memory
- Build data structures like linked lists and trees

Although they may seem unusual at first, pointers give programmers greater control over how data is stored and accessed.

Memory and Addresses

Every variable occupies a location in memory.

Example: `int num = 50;`

Memory representation:

Address	Value
1000	50

The computer keeps track of both:

- The value (50)
- The address (1000)

Normally we work only with values, but pointers allow us to work with addresses as well.

Address Operator (&)

The address operator & is used to obtain the memory address of a variable.

Syntax: `&variableName`

Example:

```
int num = 50;
cout << &num;
```

Output: 1000

(The actual address will vary on each computer.)

Why Use &? The & operator answers the question: "Where is this variable stored in memory?"

Pointer Declaration

A pointer must be declared before it can store an address.

Syntax: `dataType *pointerName;`

Example: `int *ptr;`

This creates a pointer capable of storing the address of an integer variable.

Assigning an Address to a Pointer

```
int num = 50;
int *ptr = &num;
```

Here: ptr stores address of num

Dereferencing Operator (*)

The * operator has another important role. When used with a pointer variable, it accesses the value stored at the address. This process is called dereferencing.

Syntax: *pointerName

Example:

```
int num = 50;
int *ptr = &num;
cout << *ptr;
```

Output: 50

Understanding Dereferencing

Consider:

```
int num = 50;
int *ptr = &num;
```

Memory:

Address	Value
1000	50

Pointer: ptr = 1000

When we write: *ptr

C++ reads: "Go to address 1000 and give me the value stored there."

Result: 50

Pointers and Arrays

Pointers and arrays are closely related in C++. The name of an array itself represents the address of its first element.

Example: int numbers[5] = {10, 20, 30, 40, 50};

Memory:

```
numbers[0] = 10
numbers[1] = 20
numbers[2] = 30
numbers[3] = 40
numbers[4] = 50
```

The array name: numbers

contains the address of: numbers[0]

Example: cout << numbers;

Displays the address of the first element.

Accessing First Element Using Pointer Idea

cout << *numbers;
Output: 10
Because:
numbers → address of first element
*numbers → value of first element

Notes to Remember

1. A Pointer Stores an Address
   Normal variables store values, pointers store locations.
2. & Gives the Address
   &num returns the memory address of num.
3. * Accesses the Value
   *ptr returns the value stored at the address.
4. Pointer Type Must Match Data Type
   int *ptr; stores addresses of integers.
5. Array Name Represents an Address
   arr is the address of the first element.

Closing Thought

Pointers introduce a new way of thinking about programs. Until now, we focused on values. With pointers, we begin thinking about where those values live in memory. Although pointers can seem intimidating at first, understanding the address operator (&), dereferencing operator (*), and the relationship between pointers and arrays provides the foundation for many advanced C++ concepts that follow.

Lesson Program: Pointers in C++
Source Code: Pointers1.cpp

```cpp
#include <iostream>
using namespace std;

int main() {
    int num = 50;

    cout << "Value of num: " << num << endl;
    cout << "Address of num: " << &num << endl;

    return 0;
}
```

Lesson Program: Pointers in C++

Source Code: Pointers2.cpp

```cpp
#include <iostream>
using namespace std;

int main() {
    int num = 100;

    // Pointer storing address of num
    int *ptr = &num;

    cout << "Value of num: " << num << endl;
    cout << "Address stored in ptr: " << ptr << endl;

    return 0;
}
```

Lesson Program: Pointers in C++

Source Code: Pointers3.cpp

```cpp
#include <iostream>
using namespace std;

int main() {
    int marks = 95;
    int *ptr = &marks;

    cout << "Value of marks: " << marks << endl;
    cout << "Address of marks: " << ptr << endl;
    cout << "Value using pointer: " << *ptr << endl;

    return 0;
}
```

Lesson Program: Pointers in C++

Source Code: Pointers4.cpp

```cpp
#include <iostream>
using namespace std;

int main() {
    int salary = 25000;
    int *ptr = &salary;

    cout << "Before Change: " << salary << endl;

    // Changing value through pointer
    *ptr = 30000;

    cout << "After Change: " << salary << endl;

    return 0;
}
```

Lesson Program: Pointers in C++
Source Code: Pointers5.cpp

```cpp
#include <iostream>
using namespace std;

int main() {

    int numbers[5] = {10, 20, 30, 40, 50};

    cout << "Address of first element: " << numbers << endl;
    cout << "First element using array: " << numbers[0] << endl;
    cout << "First element using pointer: " << *numbers << endl;

   int *ptr = numbers;
   cout << "Array Elements:\n";

   for (int i = 0; i < 5; i++) {
       cout << *(ptr + i) << " ";
    }

    return 0;
}
```

Functions in C++

As programs grow larger, writing all the code inside main() becomes difficult to manage. A program may contain hundreds or even thousands of lines of code. Repeating the same logic again and again makes programs longer, harder to read, and more difficult to maintain.

To solve this problem, C++ provides **functions**.

Functions allow us to divide a large program into smaller, manageable pieces. Each function performs a specific task, making programs more organized, reusable, and easier to understand.

What is a Function?

A function is a named block of code designed to perform a particular task. Once a function is created, it can be called whenever needed.

Think of a function as a machine in a factory 🏭:

- Input goes in
- Work is performed
- Output comes out

For example:

- A calculator has an addition function
- A website has a login function
- A game has a score calculation function

Each task can be separated into its own function.

Why Do We Need Functions?

Without functions:

```
int main() {
   // code
   // code
   // code
   // hundreds of lines
}
```

Programs become difficult to read and maintain. Functions help by:

- Breaking large programs into smaller parts
- Reusing code
- Improving readability
- Making debugging easier
- Reducing repetition

How Functions Work

A function is usually created once and used many times. The general process is:

1. Define a function
2. Call the function
3. Execute the function
4. Return control to the caller

Example: `greet();`
When called, execution jumps to the function, performs its task, and returns back.

Types of Functions in C++

Functions are broadly classified into:

1. Library Functions

Predefined functions provided by C++ libraries.

Examples:

```
sqrt()
pow()
getline()
```

These functions are already written by C++ developers.

2. User-Defined Functions

Functions created by programmers.

Example:

```
void greet() {
   cout << "Welcome";
}
```

Most of our focus will be on user-defined functions.

Function Declaration and Definition

Creating a function generally involves two steps:

1. Declaration
2. Definition

Function Declaration

A declaration tells the compiler: "A function with this name and parameters exists."

It contains:

- Return type
- Function name
- Parameters

Syntax: returnType functionName(parameters);

Example: void greet();

Function Definition

The definition contains the actual code that executes.

Syntax:

```
returnType functionName(parameters) {
   // code
}
```

Example:

```
void greet() {
   cout << "Welcome to C++";
}
```

Calling a Function: A function executes only when called.

Example: greet();

Parameters and Return Types

Functions often need data to work with. This data is passed using **parameters**.

Parameters: Parameters are variables declared in the function definition that receive values.

Example:

```
void displayAge(int age) {
   cout << age;
}
```

Here: int age is a parameter.

Arguments: The actual values passed to a function are called arguments.

```
   displayAge(21);
```

Here: 21 is an argument.

Return Types: A function can send a value back using return.

Example:

```
int add(int a, int b) {
   return a + b;
}
```

Void Functions: If a function does not return anything:

```
void greet() {
   cout << "Hello";
}
```

The return type becomes: void

Call by Value

By default, C++ uses **Call by Value**. In call by value: A copy of the variable is passed to the function. Changes made inside the function do not affect the original variable.

Example:

```
void update(int num) {
   num = 100;
}
```

Calling:

```
int value = 50;
update(value);
```

After execution: value = 50

The original value remains unchanged. This happens because only a copy was modified.

Inline Functions

Normally, when a function is called:

1. Control jumps to the function
2. Function executes
3. Control returns back

This process creates a small overhead. For very small functions, C++ provides **inline functions**. An inline function asks the compiler to replace the function call with the actual function code.

Syntax:

```
inline returnType functionName(parameters) {
   // code
}
```

Example:

```
inline int square(int n) {
   return n * n;
}
```

Usage: square(5);

The compiler may directly substitute: 5 * 5 instead of performing a function call.

Advantages of Inline Functions

- Faster execution
- Reduced function call overhead
- Useful for small functions

Default Arguments

Sometimes a function parameter has a value that is commonly used. Instead of passing it every time, C++ allows default values.

Syntax: returnType functionName(type parameter = value);
Example:

```
void greet(string name = "Guest") {
    cout << "Welcome " << name;
}
```

Calling: greet();
Output: Welcome Guest

Calling: greet("Aditya");
Output: Welcome Aditya

Summary of Function Components

A function usually consists of:

Component	Purpose
Function Name	Identifies the function
Return Type	Specifies returned value
Parameters	Receives input
Function Body	Contains code
Return Statement	Sends value back

Notes to Remember

1. Every C++ program starts from main().
2. Functions help avoid code repetition.
3. Parameters receive values.
4. Arguments supply values.
5. void means no value is returned.
6. Call by Value passes copies of variables.
7. Inline functions are best for small tasks.
8. Default arguments make function calls simpler.

Closing Thought

Functions are one of the most important building blocks in programming. They transform programs from long lists of instructions into organized collections of reusable tasks. Once you start thinking in terms of functions, you begin designing programs as systems of cooperating parts rather than a single block of code. Functions are where programming starts becoming structured, modular, and professional.

Lesson Program: Functions in C++

Source Code: Functions1.cpp

```cpp
#include <iostream>
using namespace std;

// Function Declaration
void greet();

int main() {
    cout << "Inside main()\n";

    // Function Call
    greet();
    cout << "Back to main()\n";

    return 0;
}

// Function Definition
void greet() {
    cout << "Welcome to C++ Functions!\n";
}
```

Lesson Program: Functions in C++

Source Code: Functions2.cpp

```cpp
#include <iostream>
using namespace std;

// Function Definition
void displayStudent(string name, int age) {
    cout << "Name: " << name << endl;
    cout << "Age: " << age << endl;
}

int main() {
    // Passing Arguments
    displayStudent("Aditya", 23);

    return 0;
}
```

Lesson Program: Functions in C++

Source Code: Functions3.cpp

```cpp
#include <iostream>
using namespace std;

// Function returning an integer
int add(int a, int b) {
    return a + b;
}

int main() {
    int result;
    result = add(10, 20);

    cout << "Sum = " << result << endl;

    return 0;
}
```

Lesson Program: Functions in C++

Source Code: Functions4.cpp

```cpp
#include <iostream>
using namespace std;

// Function that returns nothing
void showMessage() {
    cout << "Learning Functions in C++" << endl;
}

int main() {
    showMessage();

    return 0;
}
```

Lesson Program: Functions in C++

Source Code: Functions5.cpp

```cpp
#include <iostream>
using namespace std;

void changeValue(int num) {
    cout << "Inside Function Before Change: " << num << endl;
    num = 100;
    cout << "Inside Function After Change: " << num << endl;
}

int main() {
    int value = 50;

    cout << "Before Function Call: " << value << endl;
    changeValue(value);
    cout << "After Function Call: " << value << endl;

    return 0;
}
```

Lesson Program: Functions in C++

Source Code: Functions6.cpp

```cpp
#include <iostream>
using namespace std;

// Inline Function
inline int square(int num) {
    return num * num;
}

int main() {
    cout << "Square of 5 = "<< square(5) << endl;
    cout << "Square of 10 = "<< square(10) << endl;

    return 0;
}
```

Lesson Program: Functions in C++

Source Code: Functions7.cpp

```cpp
#include <iostream>
using namespace std;

// Function with default argument
void greet(string name = "Guest") {
    cout << "Welcome "<< name << endl;
}

int main() {
    // Uses default value
    greet();

    // Uses supplied value
    greet("Aditya");
    greet("Rahul");

    return 0;
}
```

Lesson Program: Functions in C++

Source Code: Functions8.cpp

```cpp
#include <iostream>
using namespace std;

// Function Declarations
void displayLine();
int add(int a, int b);
int multiply(int a, int b);

int main() {

    displayLine();
    cout << "Addition = " << add(10, 20) << endl;
    cout << "Multiplication = " << multiply(10, 20) << endl;
    displayLine();

    return 0;
}

// Function Definitions
void displayLine() {
    cout << "--------------------\n";
}

int add(int a, int b) {
    return a + b;
}

int multiply(int a, int b) {
    return a * b;
}
```

// Recursion in C++

Until now, we have learned that functions can call other functions. But in C++, a function can also call **itself**. This technique is known as **recursion**. Recursion is one of the most powerful concepts in programming because it allows complex problems to be solved by breaking them into smaller versions of the same problem.

Many real-world situations follow a recursive pattern:

- Looking at two mirrors facing each other
- Russian nesting dolls
- Folder structures inside folders
- Family trees

In each case, a structure contains a smaller version of itself. Recursion works in a similar way.

What is Recursion?

Recursion is a programming technique in which a function calls itself directly or indirectly. Instead of solving a large problem at once, the function repeatedly solves smaller instances of the same problem until a stopping condition is reached.

Definition: Recursion is the process in which a function calls itself to solve a problem.

Why Do We Need Recursion?

Some problems are naturally recursive and become easier to solve using recursion.

Examples:

- Factorial calculation
- Fibonacci series
- Tree traversal
- Directory navigation
- Mathematical sequences

Without recursion, some solutions become longer and harder to understand. Recursion often produces cleaner and more elegant code.

How Recursion Works

Every recursive function contains two important parts:

1. Base Case

The condition that stops recursion. Without a base case, the function would continue calling itself forever.

2. Recursive Case

The part where the function calls itself. This reduces the problem into a smaller version.

General Structure of a Recursive Function

```
returnType functionName(parameters) {
   if (baseCondition) {
      return value;
   }
   return functionName(smallerProblem);
}
```

Understanding Recursion with an Example

Suppose we want to count down from 5 to 1.

Traditional Thinking

5
4
3
2
1

Recursive Thinking

```
print(5)
   print(4)
      print(3)
         print(2)
            print(1)
```

Each function call creates another function call until the base case is reached.

Recursive Function Calls

Consider: `count(3);`
Execution flow:

```
count(3)
   count(2)
      count(1)
         count(0)
```

When the base case is reached:

count(0) returns
count(1) returns
count(2) returns
count(3) returns

The function calls are completed in reverse order.

Types of Recursion

1. Direct Recursion

A function directly calls itself.

Example:

```
void display() {
   display();
}
```

Here, the function calls itself directly.

2. Indirect Recursion

A function calls another function, which eventually calls the first function.

Example:

```
void A() {
   B();
}

void B() {
   A();
}
```

Here, recursion occurs indirectly.

Advantages of Recursion

1. Simpler Code

Many complex problems become easier to write.

2. Natural Problem Solving

Some problems naturally follow recursive patterns.

3. Easier to Understand

Recursive solutions often closely match mathematical definitions.

Disadvantages of Recursion

1. More Memory Usage

Each function call requires stack space.

2. Slower Execution

Function calls create overhead.

3. Risk of Infinite Recursion

If the base case is missing, recursion never stops.

Example:

```
void loop() {
   loop();
}
```

This eventually causes a **stack overflow**.

Recursion vs Iteration

Recursion	Iteration
Uses function calls	Uses loops
Requires base case	Requires loop condition
Uses more memory	Uses less memory
Often simpler for certain problems	Usually faster

When Should We Use Recursion?

Recursion is useful when:

- The problem can be divided into smaller similar problems
- Tree-like structures are involved
- Mathematical definitions are recursive
- The recursive solution is clearer than loops

Examples:

- Factorial
- Fibonacci Series
- Binary Search
- Tree Traversal
- Tower of Hanoi

Lesson Program: Recursion in C++

Source Code: Recursion1.cpp

```cpp
#include <iostream>
using namespace std;

// Recursive Function
void countDown(int n) {
    // Base Case
    if (n == 0) {
        return;
    }

    cout << n << " ";

    // Recursive Call
    countDown(n - 1);
}

int main() {
    int num;

    cout << "Enter a number: ";
    cin >> num;

    cout << "Countdown: ";

    countDown(num);

    return 0;
}
```

Lesson Program: Recursion in C++

Source Code: Recursion2.cpp

```cpp
#include <iostream>
using namespace std;

// Recursive Function
int sum(int n) {
    // Base Case
    if (n == 1) {
        return 1;
    }

    // Recursive Case
    return n + sum(n - 1);
}

int main() {
    int num;

    cout << "Enter a number: ";
    cin >> num;
    cout << "Sum = " << sum(num);

    return 0;
}
```

OOPs in C++

Until now, we have written programs using variables, operators, arrays, strings, functions, and pointers. This approach works well for small programs, but as software becomes larger and more complex, managing code becomes difficult.

Imagine developing:

- A Banking System
- A School Management System
- An E-Commerce Website
- A Social Media Application

These applications contain thousands of lines of code and many different types of data. To organize such programs more effectively, C++ provides **Object-Oriented Programming (OOP)**.

OOP is a programming approach that organizes programs around **objects** rather than individual functions and variables.

What is OOP?

Object-Oriented Programming (OOP) is a programming paradigm that combines data and functions into a single unit called an **object**.

In simple terms: OOP allows us to represent real-world entities inside a program.
For example:

A Car has:

- Brand
- Model
- Color

and performs actions like:

- Start
- Stop
- Accelerate

Using OOP, we can represent a car as an object containing both its properties and behaviors.

Why Do We Need OOP?

As programs grow larger:

- Code becomes difficult to manage
- Data may be modified accidentally
- Reusing code becomes harder
- Maintenance becomes more complex

OOP helps by:

- Organizing code into logical units
- Improving code readability
- Protecting data
- Encouraging code reuse
- Making programs easier to maintain

Procedural Programming vs Object-Oriented Programming

Before OOP, most programs were written using a procedural approach.

Examples:

- C Language
- Early programming systems

Procedural programming focuses on functions, while OOP focuses on objects.

Procedural Programming:

In procedural programming:

- Data and functions are separate
- Functions operate on data
- Programs are divided into procedures

Example:

```
inputData();
calculateResult();
displayResult();
```

The focus is on the sequence of tasks.

Object-Oriented Programming

In OOP:

- Data and functions are grouped together
- Objects interact with each other
- Programs are built around real-world entities

Example:

Student: Name, Age, Marks
Display(), CalculateResult()

The focus is on the object rather than the procedure.

Comparison

Procedural Programming	Object-Oriented Programming
Focuses on functions	Focuses on objects
Data and functions are separate	Data and functions are combined
Less secure	Better data protection
Suitable for small programs	Suitable for large programs
Code reuse is limited	Better code reuse

Classes and Objects

- Classes and objects are the foundation of OOP.
- Before creating objects, we first create a class.

What is a Class?

A class is a blueprint or template used to create objects.

It defines:

- What data an object will have
- What actions an object can perform

Think of a class as a building blueprint. The blueprint describes:

- Number of rooms
- Number of floors
- Design

but it is not the actual building. Similarly, a class describes an object but is not the object itself.

Class Definition:

A class is defined using the class keyword.

Syntax**:**

```
class ClassName {
  // Data Members
  // Member Functions
};
```

Example**:**

```
class Student {
public:
  string name;
  int age;
};
```

Here:

- Student is the class name
- name and age are data members

What is an Object?

An object is an actual instance of a class. If a class is a blueprint, then an object is the real thing created from that blueprint.

Example:

Class: Student

Objects:

student1

student2

student3

Each object follows the structure defined by the class but stores its own data.

Object Creation:

Once a class is defined, objects can be created.

Syntax: `ClassName objectName;`

Example: `Student s1;`

Here:

- Student is the class
- s1 is the object

Accessing Object Members:

The dot (.) operator is used to access members of an object.

Example:

```
s1.name = "Aditya";
s1.age = 23;
```

Access Specifiers

Access specifiers determine who can access members of a class. C++ provides three access specifiers:

- public
- private
- protected

1. Public Access Specifier:

Members declared as public can be accessed from anywhere in the program.

Syntax:

```
public:
   data;
```

Example:

```
class Student {
public:
   string name;
};
```

Usage:

```
Student s1;
s1.name = "Aditya";
```

2. Private Access Specifier:

Members declared as private can only be accessed inside the class.

Syntax:

```
private:
   data;
```

Example:

```
class Student {
private:
   int marks;
};
```

Trying to access: `s1.marks = 90;` will produce an error.
Private members help protect data from accidental modification.

3. Protected Access Specifier:

Members declared as protected are similar to private members but can also be accessed by derived classes.

Syntax:

```
protected:
   data;
```

Example:

```
class Student {
protected:
   int rollNo;
};
```

Protected members become important when learning inheritance. For now, remember: Protected members are accessible inside the class and by related classes.

Comparison of Access Specifiers

Access Specifier	Inside Class	Outside Class	Derived Class
public	Yes	Yes	Yes
private	Yes	No	No
protected	Yes	No	Yes

Notes to Remember

1. OOP organizes programs around objects.
2. A class is a blueprint for creating objects.
3. An object is an instance of a class.
4. Classes contain data members and member functions.
5. Objects are created using class names.
6. The dot (.) operator is used to access object members.
7. Public members can be accessed anywhere.
8. Private members can only be accessed inside the class.
9. Protected members are mainly used with inheritance.

Closing Thought

Object-Oriented Programming is a different way of thinking about software. Instead of focusing only on functions and steps, OOP focuses on objects that represent real-world entities. Classes act as blueprints, objects bring those blueprints to life, and access specifiers help protect and organize data.

Understanding these concepts provides a strong foundation for advanced OOP topics such as constructors, inheritance, polymorphism, abstraction, and encapsulation.

Lesson Program: OOPs in C++

Source Code: OOPs1.cpp

```cpp
#include <iostream>
using namespace std;

// Defining a class
class Student {

public:
    // Data Members
    string name;
    int age;
    float marks;

};

int main() {
    // Creating an object of Student class
    Student s1;

    // Assigning values to object members
    s1.name = "Aditya";
    s1.age = 23;
    s1.marks = 85.5;

    // Displaying object data
    cout << "Student Details\n";
    cout << "Name: " << s1.name << endl;
    cout << "Age: " << s1.age << endl;
    cout << "Marks: " << s1.marks << endl;

    return 0;
}
```

Lesson Program: OOPs in C++

Source Code: OOPs2.cpp

```cpp
#include <iostream>
using namespace std;

// Defining a class
class Employee {

public:
    // Data Members
    string name;
    int salary;

    // Member Function
    void display() {
        cout << "\nEmployee Details\n";
        cout << "Name: " << name << endl;
        cout << "Salary: " << salary << endl;
    }

};

int main() {
    // Creating object
    Employee emp1;

    // Assigning values
    emp1.name = "Rahul";
    emp1.salary = 45000;

    // Calling member function
    emp1.display();

    return 0;
}
```

Lesson Program: OOPs in C++

Source Code: OOPs3.cpp

```cpp
#include <iostream>
using namespace std;

class Student {

public:
    // Accessible everywhere
    string name;

private:
    // Accessible only inside the class
    int marks;

protected:
    // Accessible inside class and derived classes
    int rollNo;

public:
    // Public member function
    void setData() {

        // Private and protected members can be accessed inside the class
        marks = 90;
        rollNo = 101;
    }

    void displayData() {
        cout << "Name: " << name << endl;
        cout << "Marks: " << marks << endl;
        cout << "Roll No: " << rollNo << endl;
    }

};
```

```cpp
int main() {
    Student s1;

    // Public member can be accessed directly
    s1.name = "Aman";

    // Setting values using member function
    s1.setData();

    // Displaying data
    s1.displayData();

    /*
    The following statements will cause errors:

    s1.marks = 90;      // Private member
    s1.rollNo = 101;    // Protected member
    */

    return 0;
}
```

// Properties of Object-Oriented Programming (OOP)

Object-Oriented Programming is more than just classes and objects. The real strength of OOP comes from a set of important principles that help programmers build software that is organized, secure, reusable, and easy to maintain.

These principles are known as the **Properties of OOP**. The four main properties of OOP are:

1. Abstraction
2. Encapsulation
3. Inheritance
4. Polymorphism

These concepts help transform simple programs into well-structured software systems.

1. Abstraction

What is Abstraction?

Abstraction means showing only the essential details while hiding the unnecessary implementation details.
In simple terms: Focus on what an object does rather than how it does it.

Real-Life Example:

Consider a car.
As a driver, you know:

- How to start the car
- How to accelerate
- How to brake

But you do not need to know:

- How fuel is injected
- How pistons move
- How the engine generates power

These internal details are hidden. This is abstraction.

Programming Perspective

Suppose a class provides:

```
startEngine();
```

The user only calls the function. The complex implementation remains hidden inside the class.

Benefits of Abstraction

- Reduces complexity
- Improves readability
- Hides unnecessary details
- Makes software easier to use

2. Encapsulation

What is Encapsulation?

Encapsulation is the process of combining data and functions into a single unit and restricting direct access to some of the data.
In simple terms: Encapsulation means wrapping data and methods together inside a class.

Real-Life Example:

Think of a medicine capsule. The medicine is enclosed inside a protective cover.
Similarly, in OOP:

- Data is protected inside a class
- Functions provide controlled access

Programming Perspective

```
class Student {
private:
   int marks;

public:
   void setMarks(int m);
   int getMarks();
};
```

Here:

- marks cannot be accessed directly
- Functions control access

Benefits of Encapsulation

- Protects data
- Improves security
- Prevents accidental modification
- Makes code easier to maintain

3. Inheritance

What is Inheritance?

Inheritance allows one class to acquire the properties and behaviors of another class. In simple terms: A new class can reuse the features of an existing class.

Real-Life Example:

A child inherits characteristics from parents.

Examples:

- Eye color
- Hair color
- Height

Similarly, classes can inherit data and functions from other classes.

Programming Perspective:

Suppose we have:

```
class Person {
public:
   string name;
   int age;
};
```

A Student class can inherit these properties:

```
class Student : public Person {
};
```

Now Student automatically gets:

name

age

without rewriting them.

Benefits of Inheritance:

- Code reuse
- Reduced duplication
- Easier maintenance
- Better organization

4. Polymorphism

What is Polymorphism?

The word polymorphism comes from:

Poly = Many
Morph = Forms

Meaning: One interface, many forms

Real-Life Example:

Consider a person. The same person can be:

- A student at college
- A customer at a store
- A player on a sports team

One person, multiple roles.

Programming Perspective:

Suppose a function: `drawShape();`
can behave differently for:

- Circle
- Rectangle
- Triangle

The same function name produces different behaviours. This is polymorphism.

Benefits of Polymorphism:

- Flexibility
- Reusability
- Cleaner code
- Easier program expansion

Relationship Between OOP Properties

These properties work together:

Abstraction: Hides complexity.
Encapsulation: Protects data.
Inheritance: Promotes code reuse.
Polymorphism: Provides flexibility.

Together, they make software:

- More modular
- More reusable
- More maintainable
- Easier to understand

Quick Summary

Property	Purpose
Abstraction	Hides implementation details
Encapsulation	Protects and bundles data
Inheritance	Reuses existing code
Polymorphism	One interface, many behaviours

Notes to Remember:

1. Abstraction hides unnecessary details.
2. Encapsulation wraps data and functions together.
3. Inheritance allows one class to acquire features of another.
4. Polymorphism allows the same interface to behave differently.
5. These four properties form the foundation of Object-Oriented Programming.

Closing Thought

The real power of OOP does not come from classes and objects alone. It comes from these four properties working together. Abstraction simplifies, encapsulation protects, inheritance reuses, and polymorphism adapts. Understanding these concepts gives you a glimpse into how large-scale software systems are designed and maintained in the real world.

Lesson Program: OOPs Properties Abstraction in C++

Source Code: Properties1.cpp

```cpp
#include <iostream>
using namespace std;

class Car {

public:
    // Public function available to user
    void startCar() {
        cout << "Starting Car...\n";

        // Internal details hidden from user
        startEngine();
    }

private:
    // Hidden implementation
    void startEngine() {
        cout << "Engine Started Successfully!\n";
    }

};

int main() {
    Car myCar;

    // User only calls startCar()
    myCar.startCar();

    return 0;
}
```

Lesson Program: OOPs Properties Encapsulation in C++

Source Code: Properties2.cpp

```cpp
#include <iostream>
using namespace std;

class Student {

private:
    int marks;

public:
    // Setter Function
    void setMarks(int m) {
        marks = m;
    }

    // Getter Function
    int getMarks() {
        return marks;
    }

};

int main() {
    Student s1;

    // Setting value using public function
    s1.setMarks(85);

    cout << "Marks: "  << s1.getMarks();

    return 0;
}
```

Lesson Program: OOPs Properties Inheritance in C++

Source Code: Properties3.cpp

```cpp
#include <iostream>
using namespace std;

// Parent Class
class Person {

public:
    string name;
    int age;

};

// Child Class
class Student : public Person {

public:
    int rollNo;

};

int main() {
    Student s1;

    // Inherited members
    s1.name = "Aditya";
    s1.age = 23;

    // Own member
    s1.rollNo = 101;

    cout << "Name: " << s1.name << endl;
    cout << "Age: " << s1.age << endl;
    cout << "Roll No: " << s1.rollNo << endl;

    return 0;
}
```

Lesson Program: OOPs Properties Polymorphism in C++

Source Code: Properties4.cpp

```cpp
#include <iostream>
using namespace std;

class Calculator {

public:
    // Function 1
    int add(int a, int b) {
        return a + b;
    }

    // Function 2
    float add(float a, float b) {
        return a + b;
    }

};

int main() {

    Calculator calc;

    cout << "Integer Addition: " << calc.add(10, 20) << endl;
    cout << "Float Addition: " << calc.add(10.5f, 20.5f) << endl;

    return 0;
}
```

Lesson Program: OOPs Properties in C++

Source Code: OOPsProperties.cpp

```cpp
#include <iostream>
using namespace std;

// Parent Class
class Person {

protected:
    string name;

public:
    void setName(string n) {
        name = n;
    }

};

// Child Class
class Student : public Person {

private:
    int marks;

public:
    void setMarks(int m) {
        marks = m;
    }

    void display() {
        cout << "Name: " << name << endl;
        cout << "Marks: " << marks << endl;
    }

};

int main() {

    Student s1;

    s1.setName("Aditya");
    s1.setMarks(90);

    s1.display();

    return 0;
}
```

Further Advance Topics in C++

Throughout this book, we focused on the core building blocks of C++. You learned how to:

- Store and manipulate data using variables
- Take input and display output
- Perform calculations using operators
- Make decisions using conditional statements
- Repeat tasks using loops
- Store collections of data using arrays and strings
- Organize programs using functions
- Understand the basics of pointers
- Build simple applications using Object-Oriented Programming

With these skills, you can already write many useful programs.

However, C++ is a vast programming language used for developing operating systems, desktop applications, game engines, embedded systems, databases, and high-performance software. As programs become larger and more complex, programmers need additional concepts that make programs faster, more flexible, and easier to maintain.

The topics in this chapter are commonly learned after mastering the fundamentals. They introduce advanced techniques and libraries that are widely used in modern C++ development. Let's briefly look at some of these advanced topics.

Structures (struct)

A structure allows you to group multiple variables of different data types into a single unit. Structures are useful when several pieces of information belong together.

Example:

```
1  struct Student {
2
3      string name;
4      int age;
5      float marks;
6
7  };
8
9  Student s1 = {"Aditya", 23, 89.5};
10
11 cout << s1.name;
```

Structures are commonly used in:

- Student Management Systems
- Banking Applications
- Employee Records
- Inventory Systems
- Game Development

Pointers (Advanced Usage)

In this course, you learned the basics of pointers. In advanced C++, pointers are used extensively for managing memory and creating dynamic data structures.

Advanced pointer applications include:

- Dynamic Memory Allocation
- Linked Lists
- Trees and Graphs
- Smart Pointers
- Operating Systems

Enumerations (enum)

Enumerations allow programmers to create meaningful names for a fixed set of constant values. Instead of using numbers like 0, 1, and 2, enums make programs more readable.

Example:

```
1  enum Day {
2
3      MONDAY,
4      TUESDAY,
5      WEDNESDAY
6
7  };
8
9  Day today = TUESDAY;
10
11 cout << today;
```

Enums are commonly used for:

- Menu Systems
- Game States
- User Roles
- Days and Months
- Status Codes

Dynamic Memory Allocation

Normally, memory is allocated when the program starts. Dynamic memory allocation allows memory to be created while the program is running. This makes programs more flexible and efficient.

Dynamic memory is used in:

- Large Applications
- Data Structures
- Game Engines
- Databases
- Memory-intensive Programs

File Handling

File handling allows programs to save data permanently inside files instead of losing it when the program ends.

Example:

```cpp
#include <fstream>

ofstream file("student.txt");

file << "Learning C++";

file.close();
```

File handling is used for:

- Saving Records
- Report Generation
- Configuration Files
- Log Files
- Databases

Templates

Templates allow us to write one function or class that works with multiple data types. Instead of writing separate functions for integers, floats, and doubles, a single template can handle them all.

Example:

```cpp
template <typename T>

T add(T a, T b) {

    return a + b;

}
```

Templates are used in:

- Generic Programming
- Standard Template Library (STL)
- Reusable Libraries
- Framework Development

Standard Template Library (STL)

The Standard Template Library (STL) is one of the most powerful features of C++. It provides ready-made classes and algorithms that make programming faster and easier.

Example:

```cpp
#include <vector>

vector<int> numbers = {10, 20, 30};

cout << numbers[0];
```

STL includes:

- Vectors
- Lists
- Queues
- Stacks
- Maps
- Sets
- Algorithms

Exception Handling

Sometimes unexpected errors occur while a program is running. Exception handling provides a safe way to detect and handle these errors without crashing the program.

Example:

```cpp
try {

    throw "Error";

}

catch (...) {

    cout << "Exception Caught";

}
```

Exception handling is useful for:

- File Operations
- User Input Validation
- Database Applications
- Network Programming

Namespaces

Namespaces help organize code and avoid naming conflicts in large projects.

Example:

```
namespace College {

    int students = 500;

}

cout << College::students;
```

Namespaces are used in:

- Large Projects
- Libraries
- Frameworks
- Team Development

Multithreading

Multithreading allows multiple tasks to run simultaneously, making programs faster and more responsive.

Example:

```
#include <thread>

void display() {

    cout << "Hello";

}

thread t(display);

t.join();
```

Multithreading is used in:

- Games
- Browsers
- Video Editing Software
- Servers
- Scientific Computing

GUI Programming

Graphical User Interface (GUI) programming allows developers to create windows, buttons, menus, and graphical applications instead of console-based programs.

Example
Popular C++ GUI frameworks include:

- Qt
- wxWidgets
- FLTK

GUI applications include:

- Desktop Software
- Media Players
- Editors
- Business Applications

Networking

Networking enables programs to communicate with other computers over a network or the internet.

Applications include:

- Chat Applications
- Multiplayer Games
- Web Servers
- Cloud Applications
- IoT Devices

Closing Note

The topics introduced here represent only a small part of what C++ has to offer. As you continue your programming journey, you'll discover that C++ is capable of building everything from simple console applications to operating systems, game engines, robotics software, financial systems, and artificial intelligence applications.

The fundamentals you have learned in this course are the foundation for all of these advanced concepts. With regular practice and curiosity, you are now well prepared to explore the exciting world of advanced

THANK YOU!!!

"A program is more than code. It is logic, creativity, curiosity, and countless small ideas working together to solve one problem."